Astra Control Service を使用

Astra Control Service

NetApp
January 25, 2022

This PDF was generated from https://docs.netapp.com/ja-jp/astra-control-service/use/log-in.html on January 25, 2022. Always check docs.netapp.com for the latest.

目次

Astra Control Service を使用 1
Astra Control Service にログインします 1
アプリを管理、保護します 1
アプリとコンピューティングの健全性を表示します 33
バケットを管理する 37
アカウントを管理します 40
アプリケーションとクラスタの管理を解除します 49

Astra Control Service を使用

Astra Control Service にログインします

Astra Control Service には、にアクセスして、 SaaS ベースのユーザインターフェイスからアクセスできます https://astra.netapp.io。

> シングルサインオンを使用して、社内ディレクトリ（フェデレーション ID ）からのクレデンシャルを使用してログインできます。詳細については、を参照してください "Cloud Central ヘルプセンター" 次に、 * Cloud Central サインインオプション * を選択します。

必要なもの

- "Cloud Central のユーザ ID"。
- "新しい Astra Control アカウント" または 。
- サポートされている Web ブラウザ。

  Astra Control Service は、最新バージョンの Firefox 、 Safari 、 Chrome をサポートしています。最低解像度は 1280 x 720 です。

手順

1. Web ブラウザを開き、に進みます https://astra.netapp.io。
2. NetApp Cloud Central のクレデンシャルを使用してログインします。

アプリを管理、保護します

アプリの管理を開始します

お先にどうぞ "Kubernetes クラスタを Astra Control に追加"では、クラスターにアプリケーションをインストールし（ Astra Control の外部）、 Astra Control の [アプリ] ページに移動して、アプリの管理を開始できます。

アプリケーション管理の要件

Astra Control アプリケーションの管理に関する次の要件を考慮してください。

- * ライセンス * ： Astra Control Center を使用してアプリケーションを管理するには、 Astra Control Center ライセンスが必要です。
- * 名前空間 * ： Astra Control では、アプリケーションが複数の名前空間にまたがることはありませんが、名前空間には複数のアプリケーションを含めることができます。
- * StorageClass* ： StorageClass が明示的に設定されたアプリケーションをインストールし、そのアプリケーションをクローニングする必要がある場合、クローン処理のターゲットクラスタに最初に指定された StorageClass が必要です。明示的に StorageClass を設定したアプリケーションを、同じストレージクラスを使用しないクラスタにクローニングすると、失敗します。
- * Kubernetes リソース * ： Astra Control で収集されない Kubernetes リソースを使用するアプリケーションには、アプリケーションデータの完全な管理機能がない可能性があります。Astra Control では、次の

Kubernetes リソースが収集されます。

- クラスタロール
- ClusterRoleBinding
- ConfigMap
- CustomResourceDefinition の場合
- CustomResource の場合
- デモンセット（ DemonSet ）
- 導入
- DeploymentConfig
- 入力
- MutingWebhook
- PersistentVolumeClaim のように表示され
- ポッド
- ReplicaSet
- RoleBinding です
- ロール
- ルート
- 秘密
- サービス
- サービスアカウント
- Stateful役立つ セット
- 検証 Webhook

サポートされているアプリインストール方法

次の方法を使用して、 Astra Control で管理するアプリケーションをインストールできます。

- * マニフェストファイル * ： Astra Control は、 kubectl を使用してマニフェストファイルからインストールされたアプリケーションをサポートします。例：

  ```
  kubectl apply -f myapp.yaml
  ```

- * Helm 3 * ： Helm を使用してアプリケーションをインストールする場合、 Astra Control には Helm バージョン 3 が必要です。Helm 3 （または Helm 2 から Helm 3 にアップグレード）を使用してインストールされたアプリケーションの管理とクローニングが完全にサポートされています。Helm 2 でインストールされたアプリケーションの管理はサポートされていません。
- * オペレータが導入したアプリケーション * ： Astra Control は、名前空間を対象とした演算子を使用してインストールされたアプリケーションをサポートします。これらの演算子は、一般に「パスバイリファレンス」アーキテクチャではなく「パスバイ値」で設計されています。これらのパターンに続くいくつかのオペレータアプリを次に示します。

- "Apache K8ssandra"
- "Jenkins CI"
- "Percona XtraDB クラスタ"

Astra Control では、「パスバイリファレンス」アーキテクチャ（ CockroachDB オペレータなど）で設計されたオペレータをクローニングできない場合があります。クローニング処理では、クローニング処理の一環として独自の新しいシークレットが存在する場合でも、クローニングされたオペレータがソースオペレータから Kubernetes シークレットを参照しようとします。Astra Control がソースオペレータの Kubernetes シークレットを認識しないため、クローニング処理が失敗する場合があります。

> インストールする演算子とアプリケーションは、同じ名前空間を使用する必要があります。このような名前空間を使用するには、演算子の deployment.yaml ファイルを変更する必要があります。

クラスタにアプリをインストールします

Astra Control にクラスタを追加したので、クラスタにアプリケーションをインストールできます。新しいストレージクラスでは、永続ボリュームはデフォルトでプロビジョニングされます。ポッドがオンラインになったら、 Astra Control を使用してアプリケーションを管理できます。

Astra Control は、 Astra Control によってインストールされたストレージクラス上にストレージがある場合にのみ、ステートフルアプリケーションを管理する。

- "GKE クラスタのストレージクラスについて説明します"
- "AKS クラスタのストレージクラスについて学習します"

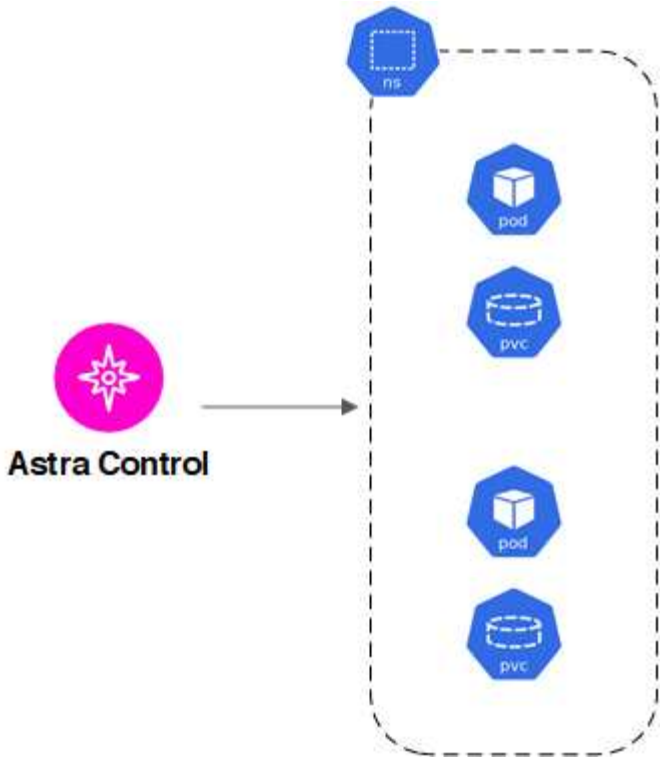
Helm チャートから一般的なアプリケーションを展開する方法については、以下を参照してください。

- "Helm チャートから MariaDB を導入します"
- "Helm チャートから MySQL を導入します"
- "Helm チャートから Postgres を導入します"
- "Helm チャートから Jenkins をデプロイします"

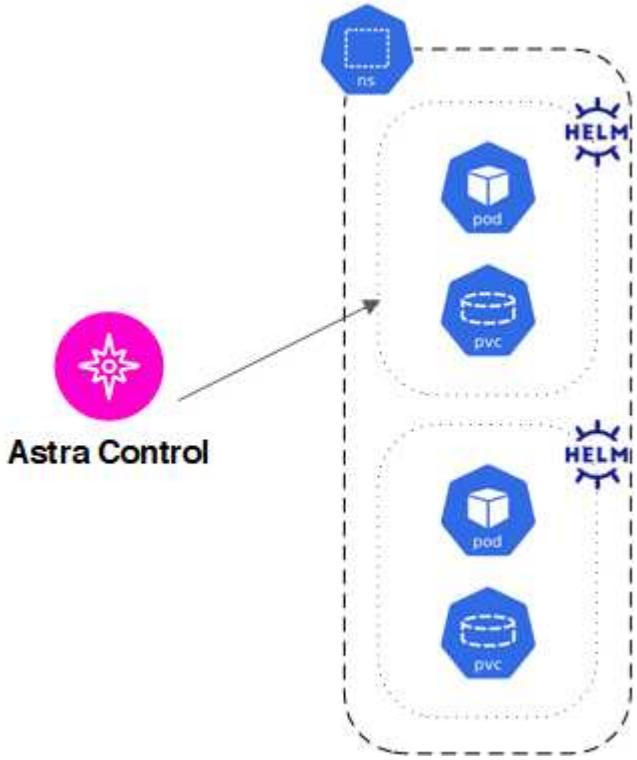
アプリの管理

Astra Control は、クラスタで実行されているアプリケーションを検出すると、管理方法を選択するまで管理対象外になります。Astra Control のマネージドアプリケーションには、次のいずれかを使用できます。

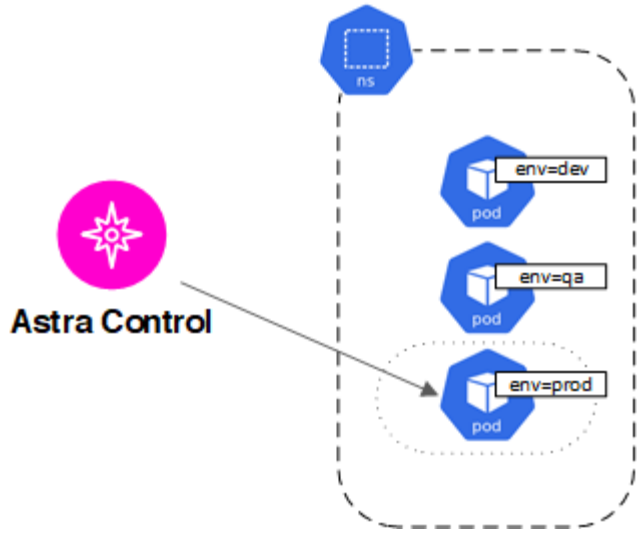
- ネームスペース。ネームスペース内のすべてのリソースを含みます



- ネームスペース内に helm3 を使用して導入された個々のアプリケーション



- Kubernetes ラベルで識別されるリソースのグループ（ Astra Control では _custom APP_in ）



以下のセクションでは、これらのオプションを使用してアプリケーションを管理する方法について説明します。

ネームスペースでアプリケーションを管理します

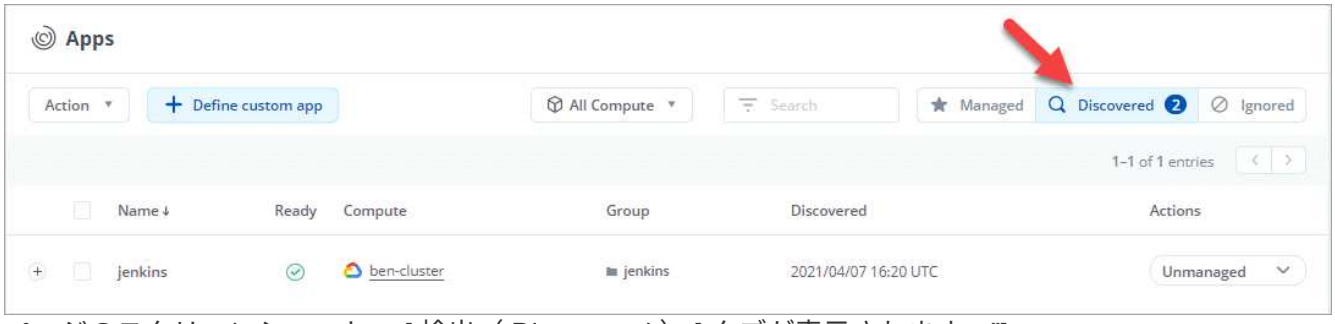
アプリページの * 検出された * セクションには、名前空間と Helm によってインストールされたアプリ、またはそれらの名前空間内のカスタムラベル付きアプリが表示されます。各アプリケーションを個別に管理することも、ネームスペースレベルで管理することもできます。データ保護処理に必要な精度のレベルが重要になります。

たとえば、毎週同じ頻度で「 Maria 」のバックアップポリシーを設定したいのに、同じネームスペースにある「 MariaDB 」をバックアップする頻度を高く設定するとします。これらのニーズに基づいて、アプリケーションを個別に管理する必要があり、単一のネームスペースで管理する必要はありません。

Astra Control を使用すると、階層の両方のレベル（名前空間とその名前空間内のアプリケーション）を個別に管理できますが、いずれか一方を選択することをお勧めします。Astra Control で実行したアクションは、ネームスペースレベルとアプリケーションレベルの両方で同時に実行される場合、失敗する可能性があります。

手順

1. 「 * アプリケーション」を選択し、「 * 検出済み * 」を選択します。



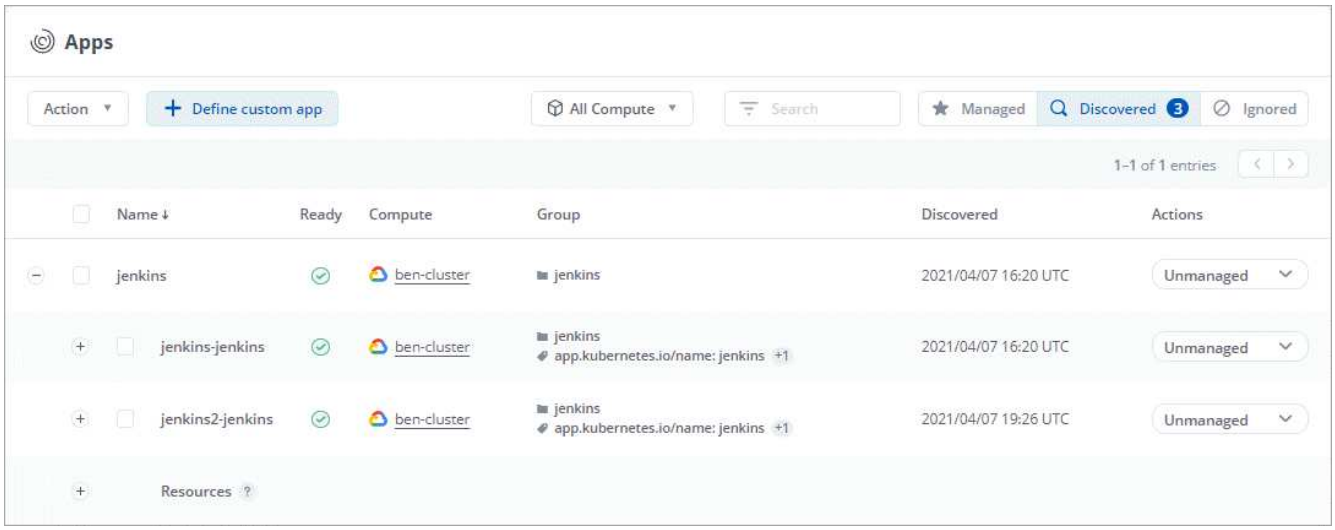
ページのスクリーンショット。 [検出（ Discovered ）] タブが表示されます。"]

2. 検出されたネームスペースのリストを表示し、ネームスペースを展開してアプリケーションと関連リソースを表示します。

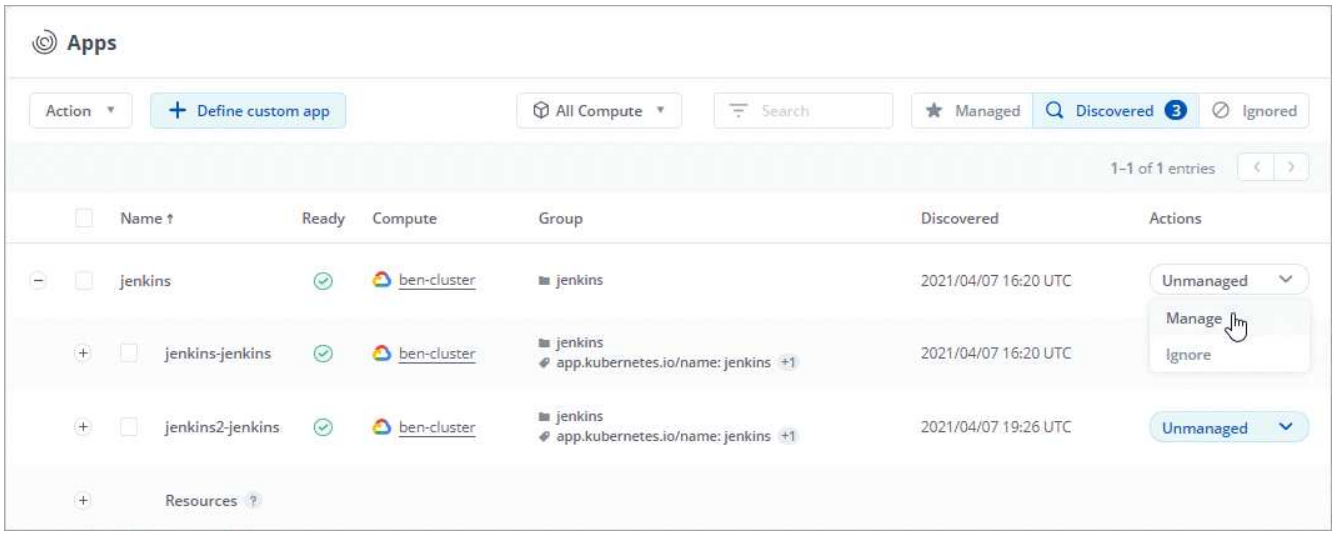
   Astra Control では、 Helm アプリケーションとカスタムラベルの付いたアプリケーションが名前空間に表

示されます。Helm ラベルがある場合は、タグアイコンで指定されます。

次に、名前空間に 2 つのアプリケーションがある例を示します。



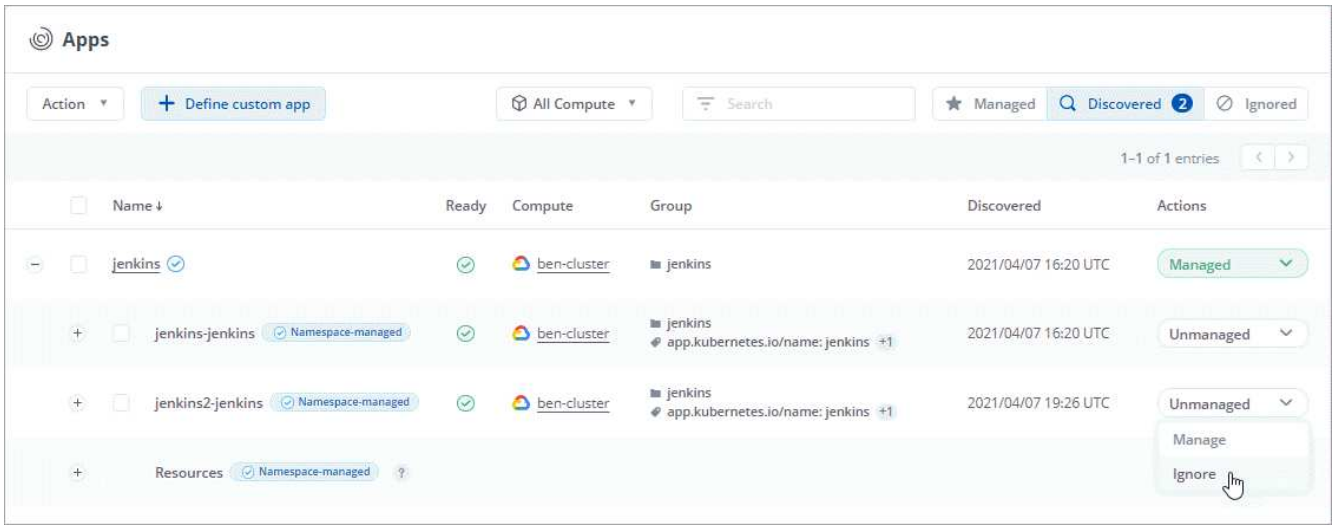
3. 各アプリケーションを個別に管理するか、ネームスペースレベルで管理するかを決定します。
4. 階層の適切なレベルで、 [* アクション * （ * Actions * ）] 列のドロップダウンリストを選択し、 [* 管理（ Manage ）] を選択します。



ページの展開されたドロップダウンリストのスクリーンショット。名前空間またはアプリを管理または無視することを選択できます。"]

5. アプリを管理しない場合は、目的のアプリの [* アクション *] 列でドロップダウンリストを選択し、 [* 無視 *] を選択します。

   たとえば、「 Jenkins 」ネームスペースですべてのアプリケーションを一緒に管理して、スナップショットポリシーとバックアップポリシーが同じになるようにする場合は、ネームスペースを管理し、ネームスペース内のアプリケーションは無視します。



ページの展開されたドロップダウンリストのスクリーンショット。名前空間またはアプリを管理または無視することを選択できます。"]

管理対象として選択したアプリは、 [管理対象 *] タブから利用できるようになりました。無視されたアプリは、 * 無視された * タブに移動します。新しいアプリケーションがインストールされると、検出されたタブにはアプリが表示されないため、見つけやすくなり、管理も簡単になります。

Kubernetes ラベルでアプリケーションを管理

Astra Control の [アプリ] ページの上部には、「 * カスタムアプリの定義 * 」という名前のアクションが含まれています。このアクションを使用して、 Kubernetes ラベルで識別されるアプリケーションを管理できます。 "Kubernetes ラベルでアプリケーションを定義する方法については、こちらをご覧ください"。

手順

1. [* アプリケーション（ * Applications ）] > [カスタムアプリケーションを定義（ Define custom app ）
2. [カスタムアプリケーションの定義 * （ Define Custom Application * ）] ダイアログボックスで、アプリケーションを管理するために必要な情報を入力します。

   a. * 新しいアプリ * ：アプリの表示名を入力します。

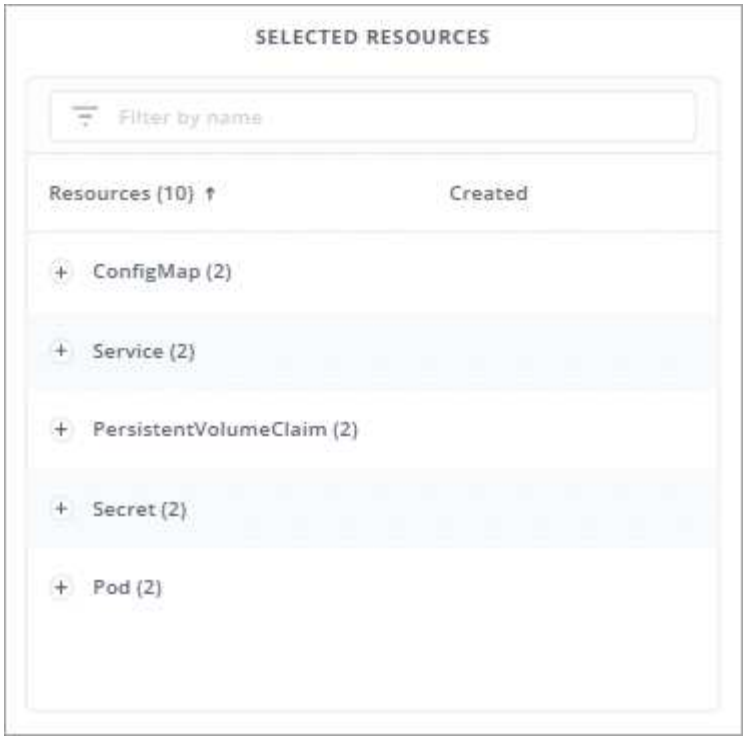
   b. * クラスタ * ：アプリケーションが存在するクラスタを選択します。

   c. * 名前空間： * アプリケーションの名前空間を選択します。

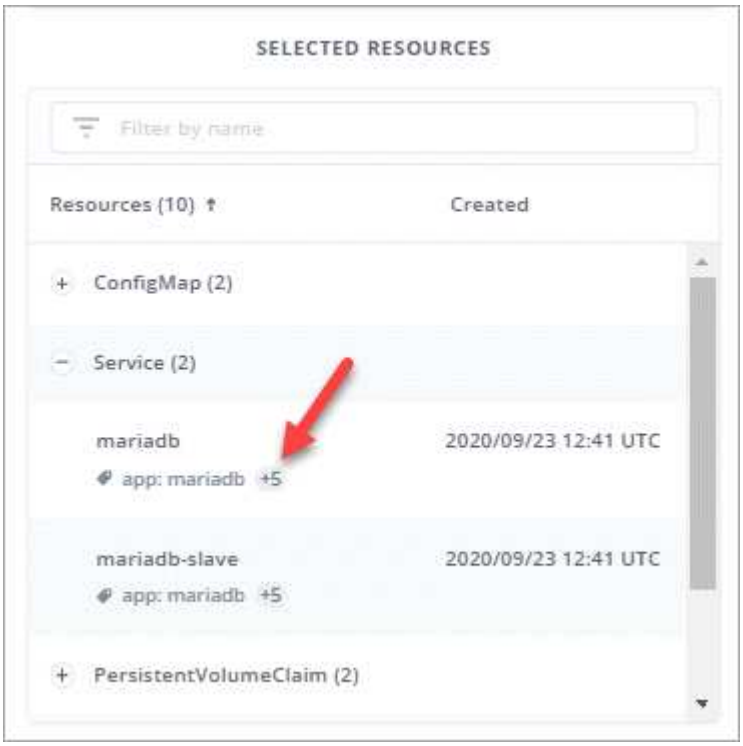
   d. * ラベル： * ラベルを入力するか、以下のリソースからラベルを選択してください。

   e. * 選択したリソース * ：保護する Kubernetes リソース（ポッド、シークレット、永続ボリュームなど）を表示および管理します。

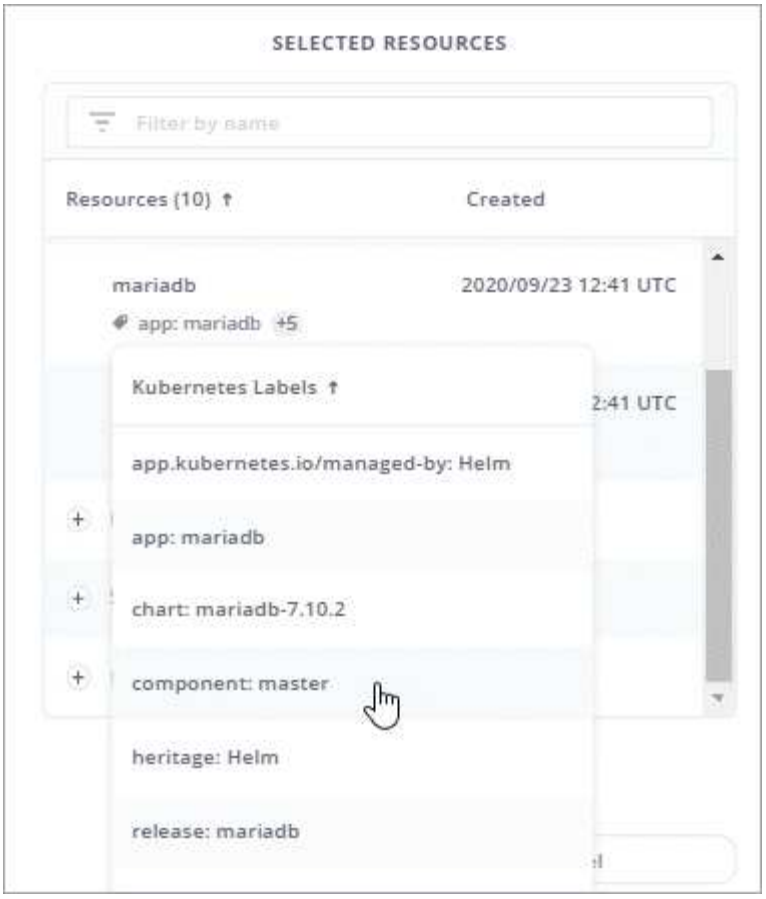
      次に例を示します。



- リソースを展開し、ラベル数を選択して、使用可能なラベルを表示します。

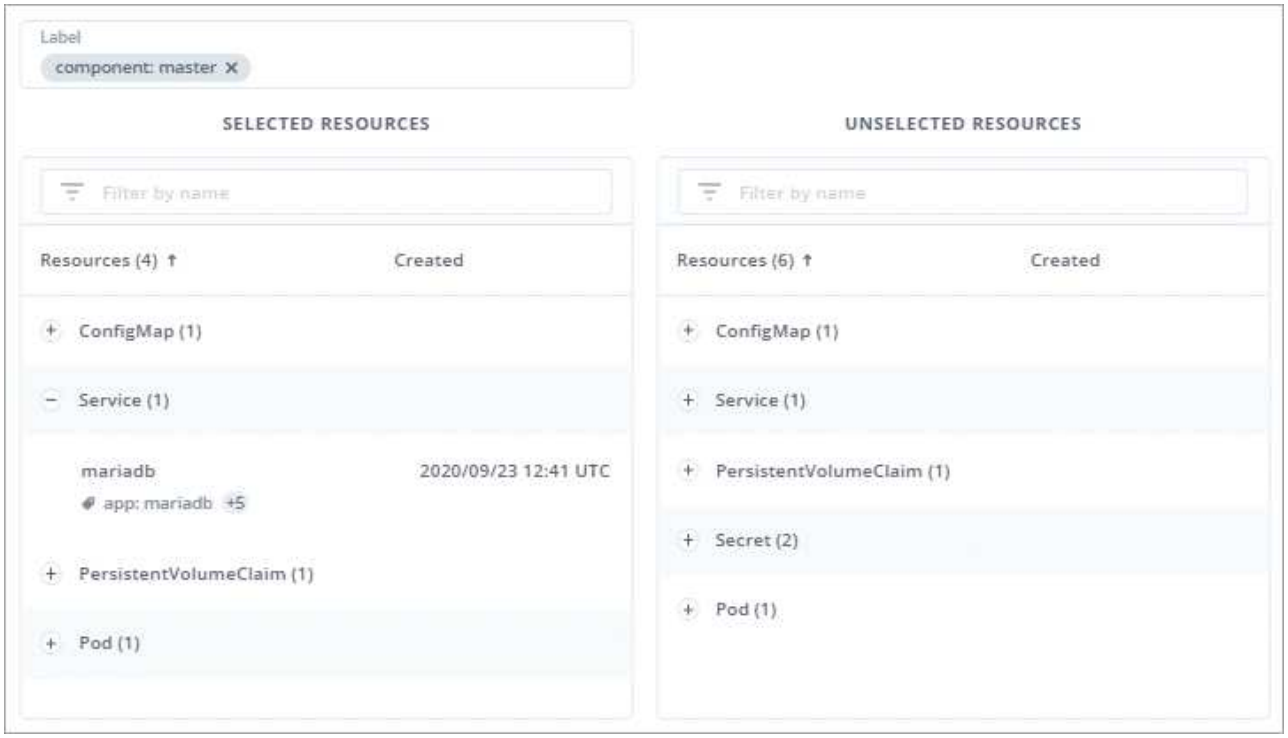


- ラベルを 1 つ選択します。



ラベルを選択すると、 [**Label**] フィールドにラベルが表示されます。Astra Control は、 [選択されていないリソース *] セクションも更新して、選択したラベルと一致しないリソースを表示します。

a. * 選択されていないリソース * ：保護する必要がないアプリケーションリソースを確認します。



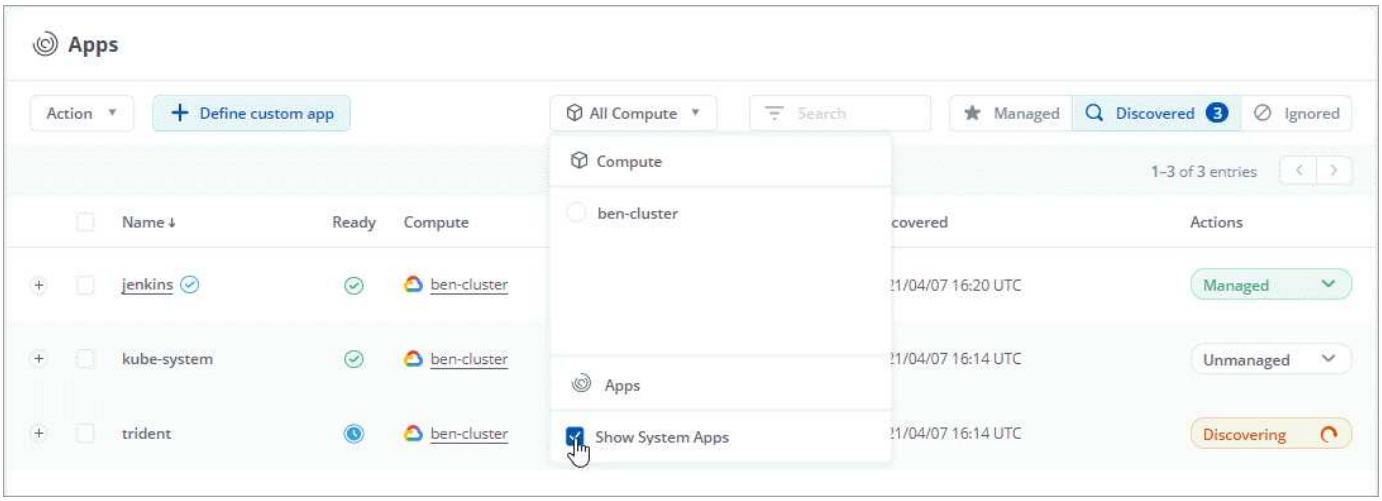
ペインでラベルを選択した後に [ラベル] フィールドにラベルを表示するスクリーンショット。"]

3. 「 * カスタムアプリを定義 * 」を選択します。

Astra Control を使用すると、アプリケーションを管理できます。これで、 [* 管理対象 * （ * Managed * ）] タブに表示されます。

システムアプリケーションについて教えてください。

Astra Control は、 Kubernetes クラスタで実行されているシステムアプリケーションも検出します。[アプリケーション（ Apps ）] リストをフィルタリングすることで、これらを表示できます。



これらのシステムアプリは、バックアップが必要になることが稀であるため、デフォルトでは表示されません。

Snapshot とバックアップでアプリケーションを保護

自動保護ポリシーまたはアドホックベースを使用してスナップショットやバックアップを作成することで、アプリケーションを保護します。

Snapshot とバックアップ

a_snapshot_は 、アプリケーションと同じプロビジョニングボリュームに格納されるアプリケーションのポイントインタイムコピーです。通常は高速です。ローカル Snapshot を使用して、アプリケーションを以前の時点にリストアします。

a_backup_ は、クラウドのオブジェクトストレージに格納されます。ローカル Snapshot に比べてバックアップに時間がかかる場合があります。ただし、クラウドのリージョン間でアクセスできるため、アプリケーションの移行が可能です。バックアップの保持期間を延長することもできます。

> _ 最新のバックアップがあるまで、完全に保護することはできません _ 。これは、永続ボリュームから離れたオブジェクトストアにバックアップが格納されるために重要です。障害や事故によってクラスタとその永続的ストレージが損傷した場合は、バックアップをリカバリする必要があります。スナップショットを使用してリカバリすることはできません。

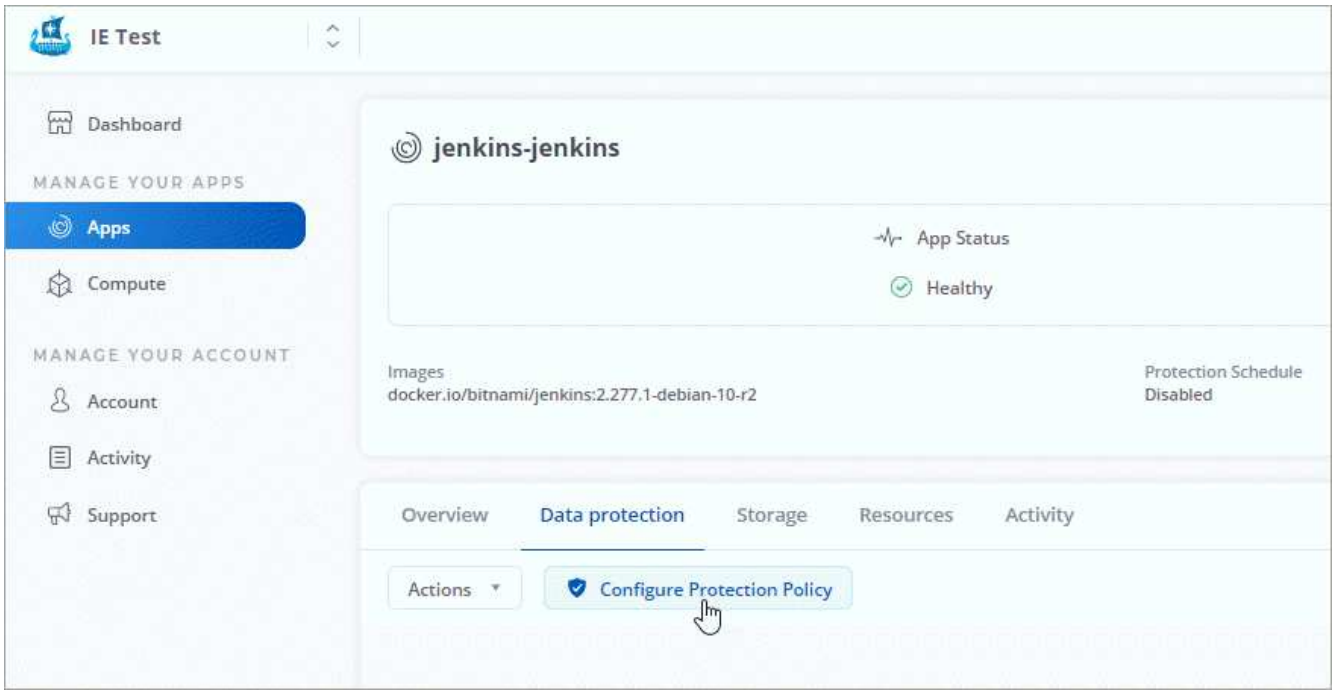
保護ポリシーを設定します

保護ポリシーは、定義されたスケジュールでスナップショット、バックアップ、またはその両方を作成することでアプリケーションを保護します。Snapshot とバックアップを毎時、日次、週次、および月単位で作成

し、保持するコピーの数を指定できます。1 時間に 1 回以上の頻度でバックアップや Snapshot を実行する必要がある場合は、次の方法があります "Astra Control REST API を使用して、スナップショットとバックアップを作成"。

手順

1. 「 * アプリケーション」を選択し、管理アプリの名前を選択します。
2. [* データ保護 *] を選択します。
3. ［ 保護ポリシーの設定 ］ を選択します。



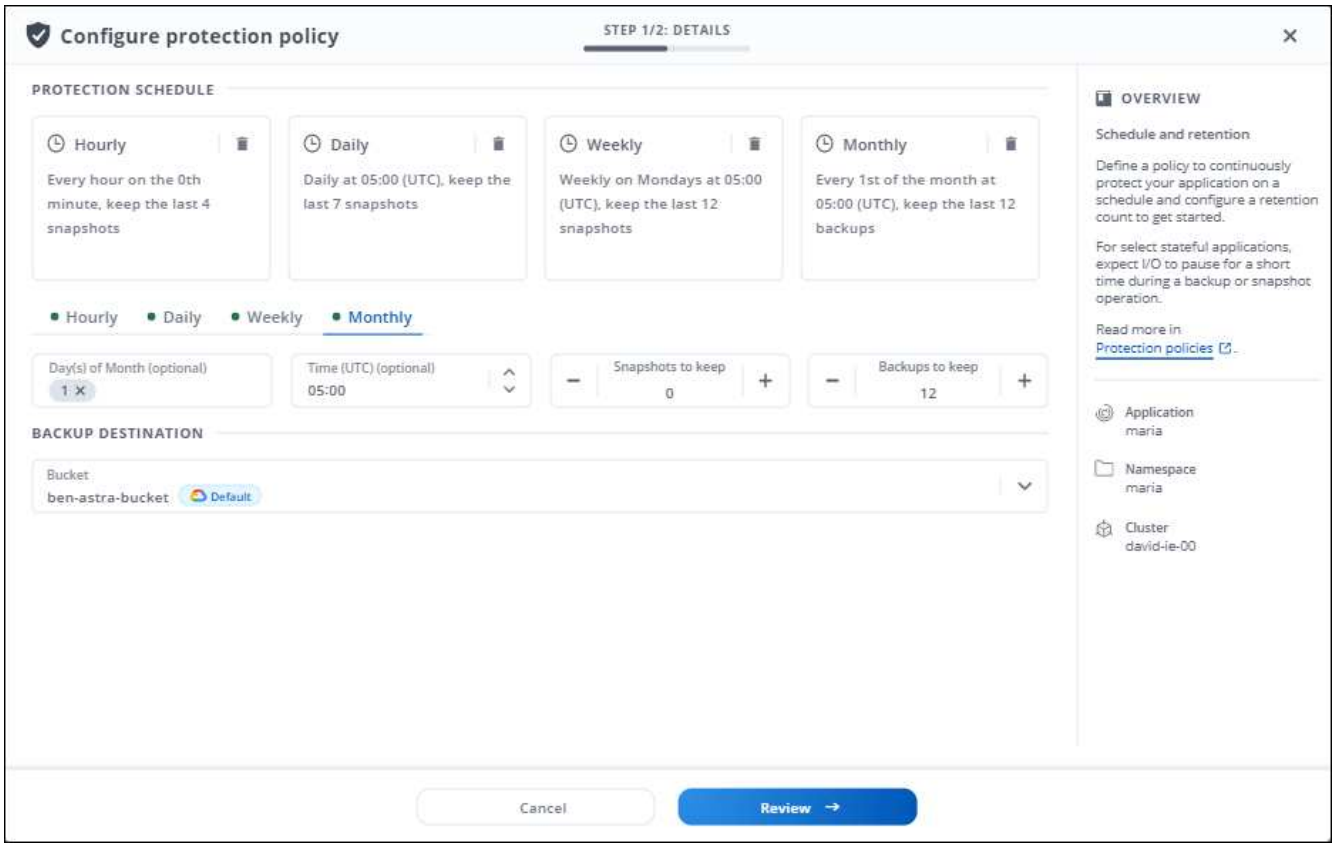
タブのスクリーンショット。保護ポリシーを設定できます。"]

4. 毎時、毎日、毎週、および毎月のスケジュールで保持する Snapshot とバックアップの数を選択して、保護スケジュールを定義します。

   スケジュールは、毎時、毎日、毎週、および毎月の各スケジュールで同時に定義できます。Snapshot とバックアップの保持レベルを設定するまで、スケジュールはアクティブになりません。

   バックアップの保持レベルを設定する際に、バックアップを格納するバケットを選択できます。

   次の例では、 Snapshot とバックアップの保護スケジュールとして、毎時、毎日、毎週、毎月の 4 つを設定します。



5. [* Review （レビュー）] を選択します
6. 「 * Configure * 」を選択します。

   以下は、これらの各手順を紹介するビデオです。

   ▶ https://docs.netapp.com/ja-jp/astra-control-service/media/use/video-set-protection-policy.mp4 *(video)*

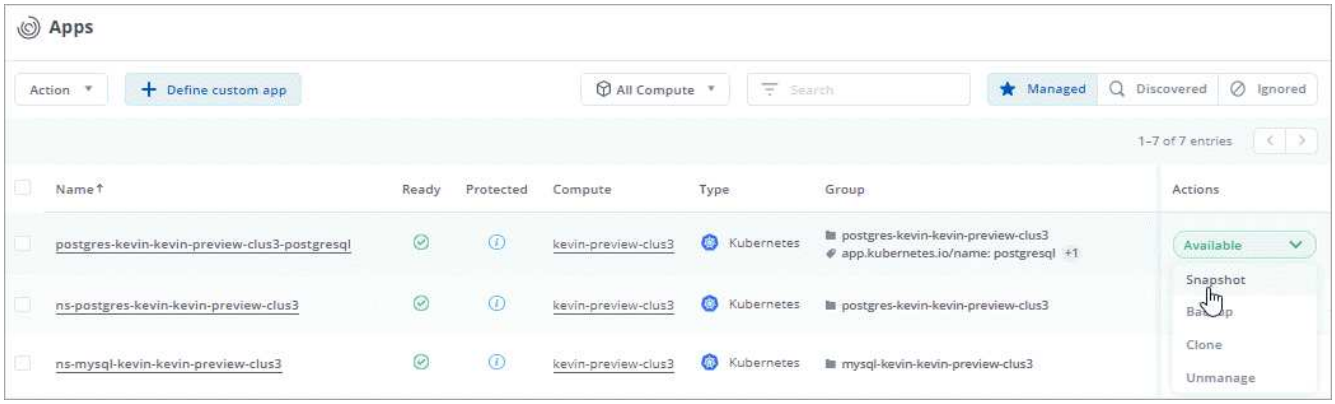
Astra Control は、定義したスケジュールと保持ポリシーを使用して、スナップショットとバックアップを作成し、保持することによって、データ保護ポリシーを実装します。

Snapshot を作成します

オンデマンド Snapshot はいつでも作成できます。

手順

1. 「 * アプリケーション * 」を選択します。
2. 目的のアプリケーションの [* アクション * （ * Actions * ）] 列でドロップダウンリストを選択します。
3. 「 * Snapshot * 」を選択します。



4. スナップショットの名前をカスタマイズし、「 * レビュー情報 * 」を選択します。
5. スナップショットの概要を確認し、 * スナップショットアプリ * を選択します。

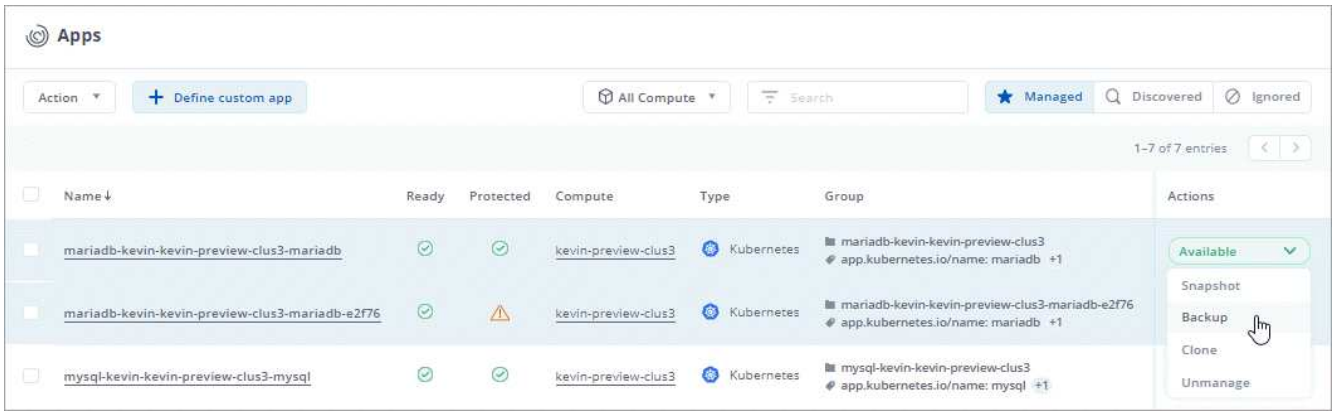
Astra Control は、アプリケーションのスナップショットを作成します。

バックアップを作成します

アプリケーションはいつでもバックアップできます。

手順

1. 「 * アプリケーション * 」を選択します。
2. 目的のアプリケーションの [* アクション * （ * Actions * ）] 列でドロップダウンリストを選択します。
3. 「 * Backup * 」を選択します。



4. バックアップの名前をカスタマイズし、既存のスナップショットからアプリケーションをバックアップするかどうかを選択し、「 * 情報の確認 * 」を選択します。
5. バックアップの概要を確認して、「 * バックアップアプリ * 」を選択します。

Astra Control ：アプリケーションのバックアップを作成

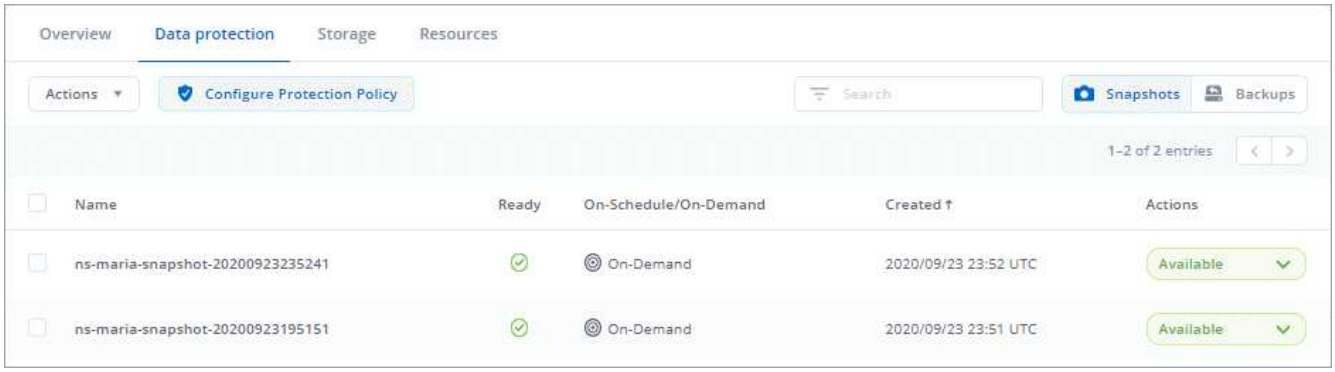
Snapshot とバックアップを表示します

アプリケーションのスナップショットとバックアップは、 [データ保護（ Data Protection ）] タブで表示できます。

手順

1. 「 * アプリケーション」を選択し、管理アプリの名前を選択します。
2. [* データ保護 *] を選択します。

   デフォルトでは、 Snapshot が表示されます。



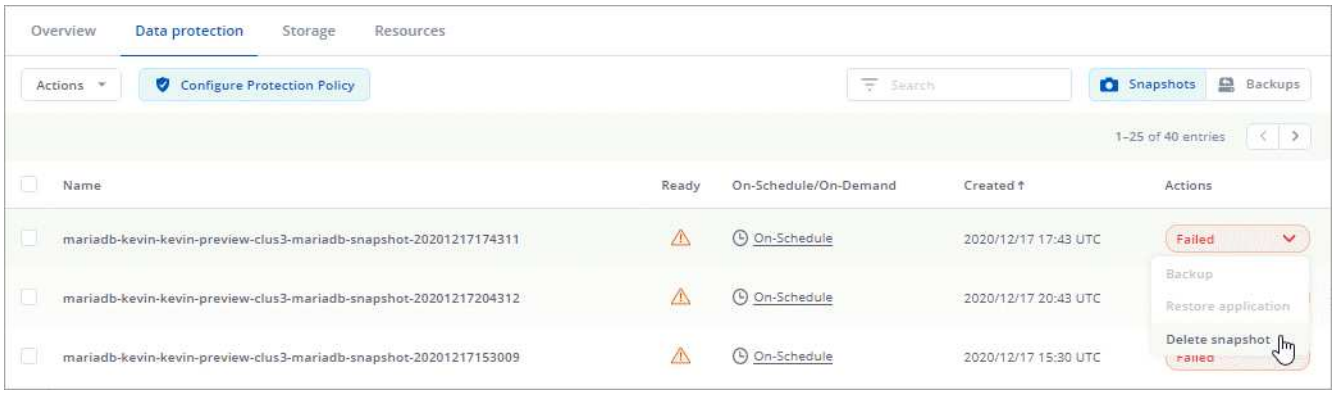
3. バックアップのリストを表示するには、「 * Backups * 」を選択します。

Snapshot を削除します

不要になったスケジュール済みまたはオンデマンドの Snapshot を削除します。

手順

1. 「 * アプリケーション」を選択し、管理アプリの名前を選択します。
2. [* データ保護 *] を選択します。
3. 目的のスナップショットの [* アクション * （ * Actions * ）] 列でドロップダウンリストを選択します。
4. 「 * スナップショットの削除 * 」を選択します。



   タブのスクリーンショット。"]

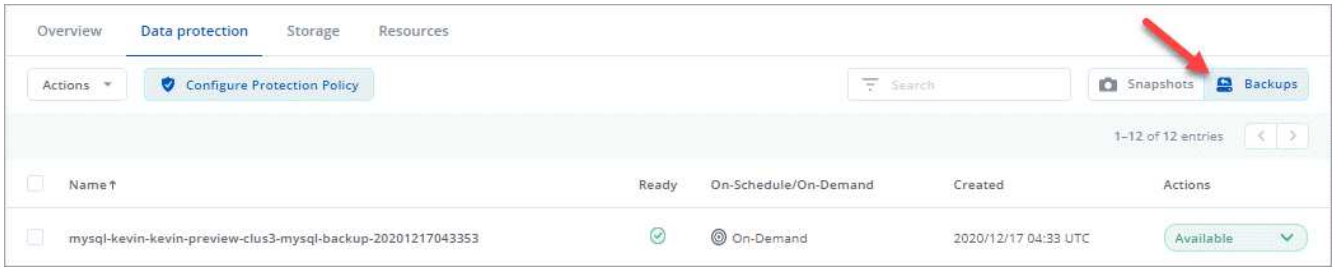
5. 削除を確認するスナップショットの名前を入力し、 * はい、スナップショットを削除 * を選択します。

Astra Control がスナップショットを削除します。

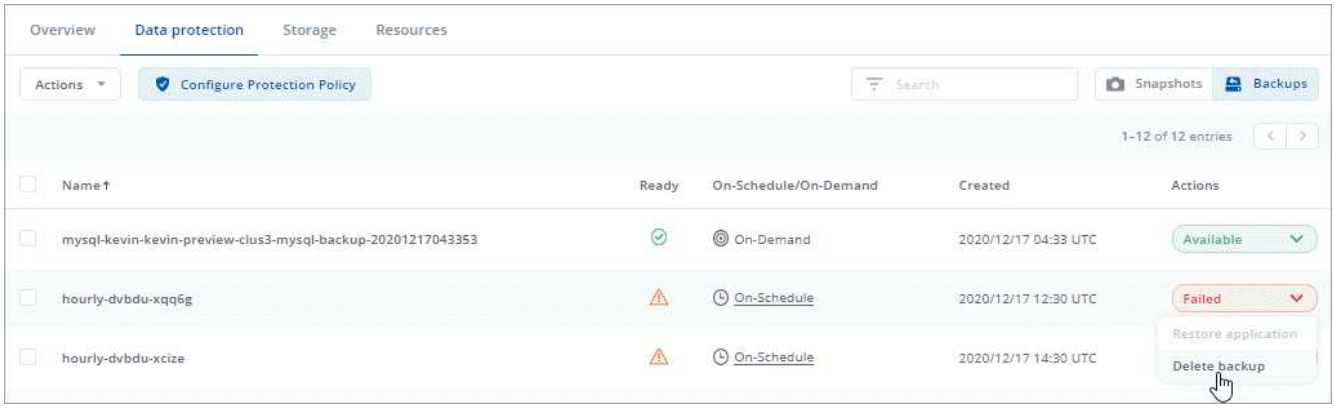
バックアップを削除します

不要になったスケジュール済みまたはオンデマンドのバックアップを削除します。

1. 「 * アプリケーション」を選択し、管理アプリの名前を選択します。
2. [* データ保護 *] を選択します。
3. 「 * Backups * 」を選択します。



4. 目的のバックアップの [* アクション *] 列のドロップダウン・リストを選択します。
5. [* バックアップの削除 *] を選択します。



タブのスクリーンショット。"]

6. 削除を確認するバックアップの名前を入力し、 * はい、バックアップを削除 * を選択します。

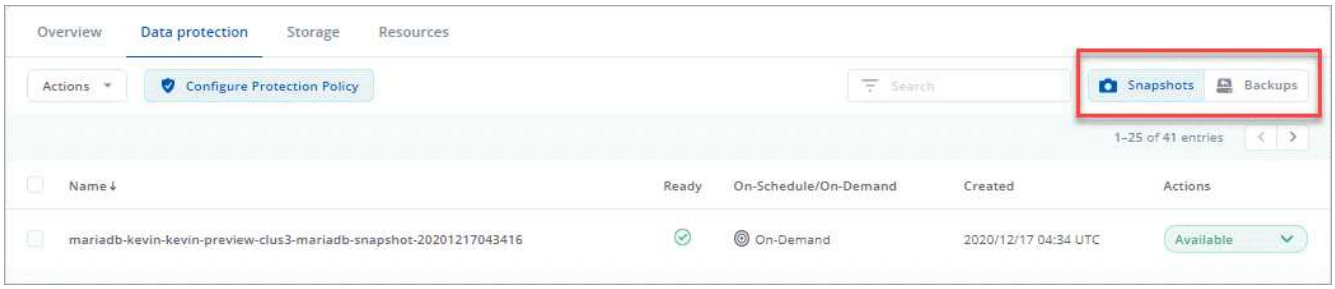
Astra Control がバックアップを削除する。

アプリケーションのリストア

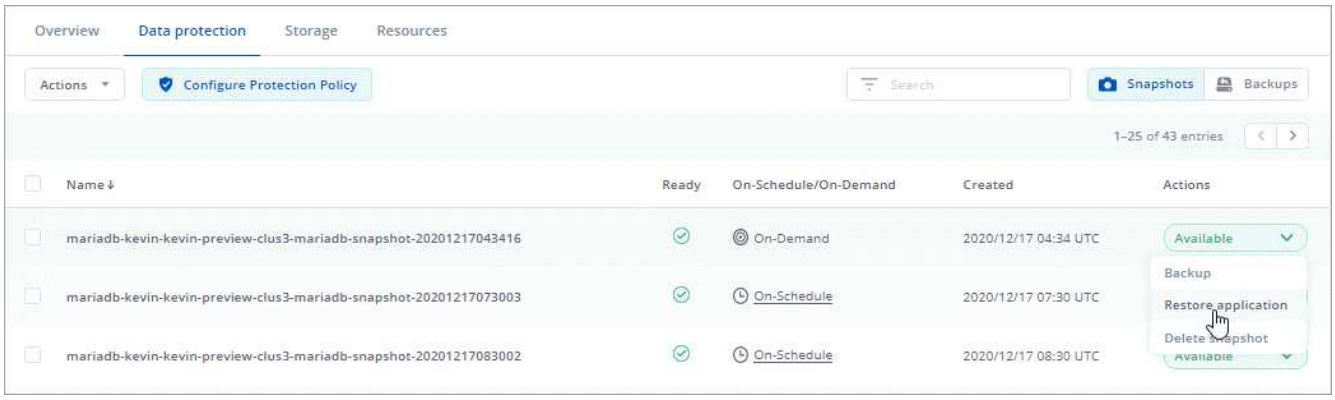
Astra Control を使用すると、スナップショットまたはバックアップからアプリケーションをリストアできます。同じクラスタにアプリケーションをリストアする場合、既存の Snapshot からのリストアは高速です。Astra Control UI またはを使用できます "Astra Control API" アプリを復元するには、

手順

1. 「 * アプリケーション」を選択し、管理アプリの名前を選択します。
2. 「 * データ保護 * 」を選択します。
3. Snapshot からリストアする場合は、「 * Snapshot * 」を選択したままにします。それ以外の場合は、「 * Backups * 」を選択してバックアップからリストアします。



4. リストアする Snapshot またはバックアップの * Actions * 列で、ドロップダウンリストを選択します。
5. [アプリケーションの復元] を選択します。



6. * リストアの詳細 * ：リストアされたアプリの詳細を指定します。アプリをインプレースで復元する（アプリを以前のバージョンのアプリに戻す）には、現在実行している名前空間クラスタとデスティネーションクラスタを選択します。
   - アプリケーションのネームスペースを入力します。
   - アプリケーションのデスティネーションクラスタを選択します。
   - [* Review （レビュー）] をクリックします
7. * リストアの概要 * ：リストア操作の詳細を確認し、「 restore 」と入力して、 * Restore * を選択します。

Astra Control は、指定した情報に基づいてアプリケーションを復元します。アプリケーションをインプレースでリストアした場合、関連付けられている永続ボリュームは削除され、再作成されます。

アプリケーションのクローン作成と移行

既存のアプリケーションをクローニングして、同じ Kubernetes クラスタまたは別のクラスタに重複するアプリケーションを作成する。Kubernetes クラスタ間でアプリケーションとストレージを移動する必要がある場合は、クローニングが役立ちます。たとえば、 CI / CD パイプラインや Kubernetes ネームスペース間でワークロードを移動できます。

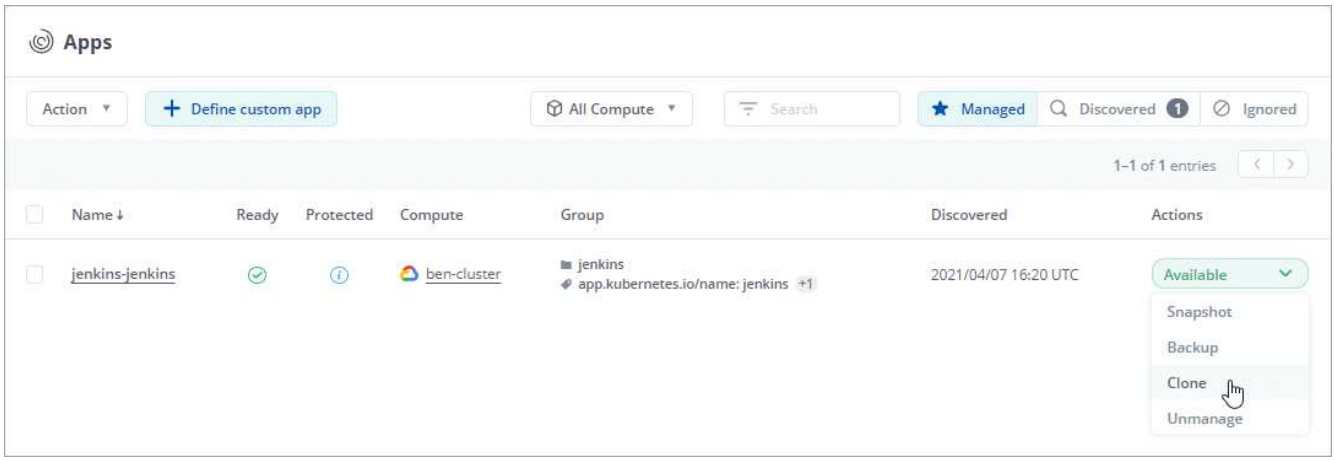
Astra Control でアプリケーションをクローニングすると、アプリケーション構成と永続的ストレージのクローンが作成されます。

オペレータが配置した Jenkins CI のインスタンスをクローニングする場合は、永続データを手動でリストアする必要があります。これは、アプリケーションの展開モデルの制限事項です。

StorageClass が明示的に設定されたアプリケーションを展開し、そのアプリケーションをクローニングする必要がある場合、ターゲットクラスタには元の StorageClass が指定されている必要があります。明示的に StorageClass を設定したアプリケーションを、同じストレージクラスを使用しないクラスタにクローニングすると、失敗します。

手順

1. 「 * アプリケーション * 」を選択します。
2. 目的のアプリケーションの * アクション * 列のドロップダウンリストを選択します。
3. 「 * Clone * 」を選択します。



4. * クローンの詳細 * ：クローンの詳細を指定します。
   - デフォルトの名前と名前空間をそのまま使用するか、編集します。
   - クローンのデスティネーションクラスタを選択してください。
   - 既存の Snapshot からクローンを作成するかバックアップを作成するかを選択します。このオプションを選択しない場合は、 Astra Control がアプリケーションの現在の状態からクローンを作成します。
5. * クローンの概要 * ：クローンの詳細を確認し、 * クローンアプリ * を選択します。

ダイアログボックスのスクリーンショット。"]

Astra Control は、入力した情報に基づいてアプリケーションをクローニングします。

アプリケーション実行フックを管理します

実行フックは、管理対象アプリケーションのスナップショットの前または後に実行できるカスタムスクリプトです。たとえば、データベースアプリケーションがある場合、実行フックを使用して、スナップショットの前にすべてのデータベーストランザクションを一時停止し、スナップショットの完了後にトランザクションを再開できます。これにより、アプリケーションと整合性のある Snapshot を作成できます。

デフォルトの実行フックと正規表現

一部のアプリケーションでは、ネットアップが提供するデフォルトの実行フックが Astra Control に付属しており、スナップショットの前後にフリーズや再開の操作を処理します。Astra Control では、正規表現を使用して、アプリケーションのコンテナイメージを次のアプリケーションに照合します。

- MariaDB
  - 正規表現 \bmariadb\b に一致しています
- MySQL
  - 正規表現 \bmysqql\b に一致しています
- PostgreSQL
  - 正規表現 \bpostgresql\b と一致します

一致した場合は、そのアプリケーションのデフォルトの実行フックがアプリケーションのアクティブな実行フックのリストに表示され、そのアプリケーションのスナップショットが作成されると、それらのフックが自動的に実行されます。カスタムアプリケーションの 1 つに、正規表現の 1 つと一致するように表示されるイメージ名が似ている場合（デフォルトの実行フックを使用しない場合）、イメージ名を変更することができます。 または、そのアプリケーションのデフォルト実行フックを無効にして、代わりにカスタムフックを使用します。

デフォルトの実行フックを削除または変更することはできません。

カスタム実行フックに関する重要な注意事項

アプリケーションの実行フックを計画するときは、次の点を考慮してください。

- Astra Control では、実行フックを実行可能なシェルスクリプトの形式で記述する必要があります。
- スクリプトサイズは 128KB に制限されています。
- Astra Control は、実行フックの設定と一致条件を使用して、スナップショットに適用できるフックを決定します。
- 実行フックの障害はすべてソフトな障害です他のフックとスナップショットは ' フックが失敗しても試行されますただし、フックが失敗すると、 * アクティビティ * ページイベントログに警告イベントが記録されます。
- 実行フックを作成、編集、または削除するには、 Owner 、 Admin 、または Member 権限を持つユーザーである必要があります。
- 実行フックの実行に 25 分以上かかる場合 ' フックは失敗し ' 戻りコードが N/A のイベント・ログ・エント

リが作成されます該当する Snapshot はタイムアウトして失敗とマークされ、タイムアウトを通知するイベントログエントリが生成されます。

> 実行フックは、実行中のアプリケーションの機能を低下させるか、完全に無効にすることが多いため、カスタム実行フックの実行時間を最小限に抑えるようにしてください。

スナップショットが実行されると、実行フックイベントが次の順序で実行されます。

1. ネットアップが提供するデフォルトの Snapshot 前実行フックは、該当するコンテナで実行されます。
2. 適用可能なカスタムスナップショット前実行フックは、適切なコンテナで実行されます。必要な数のカスタムスナップショット前フックを作成して実行できますが ' スナップショットの実行順序は保証も構成もされていません
3. スナップショットが実行されます。
4. 適用可能なカスタムスナップショット後実行フックは、適切なコンテナで実行されます。必要な数のカスタムスナップショット後フックを作成して実行できますが ' スナップショット後のこれらのフックの実行順序は保証されておらず ' 構成もできません
5. ネットアップが提供するデフォルトのポスト Snapshot 実行フックは、該当するコンテナで実行されます。

> 本番環境で実行スクリプトを有効にする前に、必ず実行フックスクリプトをテストしてください。'kubectl exec' コマンドを使用すると、スクリプトを簡単にテストできます。本番環境で実行フックを有効にしたら、作成されたスナップショットの整合性をテストします。これを行うには、アプリケーションを一時ネームスペースにクローニングし、スナップショットをリストアしてから、アプリケーションをテストします。

既存の実行フックを表示します

既存のカスタム実行フックまたはネットアップが提供するアプリケーションのデフォルト実行フックを表示できます。

手順

1. 「 * アプリケーション」に移動し、管理アプリの名前を選択します。
2. [実行フック *] タブを選択します。

   有効または無効になっているすべての実行フックを結果リストに表示できます。フックのステータス ' ソース ' および実行時（スナップショット前またはスナップショット後）を表示できます実行フックに関連するイベントログを表示するには、左側のナビゲーション領域の * アクティビティ * ページに移動します。

カスタム実行フックを作成します

アプリケーションのカスタム実行フックを作成できます。を参照してください "実行フックの例" フックの例を参照してください。実行フックを作成するには、 Owner 、 Admin 、または Member のいずれかの権限が必要です。

> 実行フックとして使用するカスタムシェルスクリプトを作成する場合は、 Linux コマンドを実行しているか、実行可能ファイルへの完全パスを提供している場合を除き、ファイルの先頭に適切なシェルを指定するようにしてください。

手順

1. 「 * アプリケーション」を選択し、管理アプリの名前を選択します。
2. [実行フック *] タブを選択します。
3. [* 新しいフックを追加 *] を選択します。
4. フックの詳細 * 領域で、フックを実行するタイミングに応じて、「 * Pre-Snapshot * 」または「 * Post-Snapshot * 」を選択します。
5. フックの一意の名前を入力します。
6. （オプション）実行中にフックに渡す引数を入力し、各引数を入力した後で Enter キーを押して、それぞれを記録します。
7. [* Container Images * （コンテナイメージ * ）] 領域で、アプリケーションに含まれるすべてのコンテナイメージに対してフックを実行する必要がある場合は、 [* Apply to all container images * （すべてのコンテナイメージに適用 * ）] チェックボックスを有効にします。代わりに、フックが 1 つ以上の指定されたコンテナイメージに対してのみ機能する場合は、 * Container image names to match * フィールドにコンテナイメージ名を入力します。
8. [* スクリプト * （ * Script * ）] 領域で、次のいずれかを実行します。
   - カスタムスクリプトをアップロードする。
     i. [ファイルのアップロード（ Upload file ）] オプションを選択します。
     ii. ファイルを参照してアップロードします。
     iii. スクリプトに一意の名前を付けます。
     iv. （オプション）他の管理者がスクリプトについて知っておく必要があるメモを入力します。
   - クリップボードからカスタムスクリプトを貼り付けます。
     i. クリップボードから貼り付け * オプションを選択します。
     ii. テキストフィールドを選択し、スクリプトテキストをフィールドに貼り付けます。
     iii. スクリプトに一意の名前を付けます。
     iv. （オプション）他の管理者がスクリプトについて知っておく必要があるメモを入力します。
9. [* フックを追加 *] を選択します。

実行フックを無効にします

アプリケーションのスナップショットの前または後に実行を一時的に禁止する場合は、実行フックを無効にできます。実行フックを無効にするには、 Owner 、 Admin 、または Member のいずれかの権限が必要です。

手順

1. 「 * アプリケーション」を選択し、管理アプリの名前を選択します。
2. [実行フック *] タブを選択します。
3. 無効にするフックの * Actions * ドロップダウンを選択します。
4. [**Disable**] を選択します。

実行フックを削除します

不要になった実行フックは完全に削除できます。実行フックを削除するには、 Owner 、 Admin 、または Member のいずれかの権限が必要です。

手順

1. 「 * アプリケーション」を選択し、管理アプリの名前を選択します。
2. [実行フック *] タブを選択します。
3. 削除するフックの * Actions * ドロップダウンを選択します。
4. 「 * 削除」を選択します。

実行フックの例

次の例を使用して、実行フックの構造を確認してください。これらのフックは、テンプレートまたはテストスクリプトとして使用できます。

シンプルな成功例

次に、成功し、標準出力および標準エラーにメッセージを書き込む単純フックの例を示します。

```
#!/bin/sh

# success_sample.sh
#
# A simple noop success hook script for testing purposes.
#
# args: None
#



#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}



#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
```

```
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running success_sample.sh"

# exit with 0 to indicate success
info "exit 0"
exit 0
```

シンプルな成功の例（**bash** バージョン）

次に、 bash 用に書かれた標準出力と標準エラーにメッセージを書き込む単純なフックの例を示します。

```
#!/bin/bash

# success_sample.bash
#
# A simple noop success hook script for testing purposes.
#
# args: None


#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}
```

```
#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running success_sample.bash"

# exit with 0 to indicate success
info "exit 0"
exit 0
```

単純な成功例（**zsh** バージョン）

これは、成功した単純なフックの例であり、標準出力と標準エラーに Z シェル用に記述されたメッセージを書き込みます。

```
#!/bin/zsh

# success_sample.zsh
#
# A simple noop success hook script for testing purposes.
#
# args: None
#
```

```
#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}

#
# main
#

# log something to stdout
info "running success_sample.zsh"

# exit with 0 to indicate success
info "exit 0"
exit 0
```

引数を指定した成功の例

次の例は、フックで args を使用する方法を示しています。

```
#!/bin/sh

# success_sample_args.sh
```

```
#
# A simple success hook script with args for testing purposes.
#
# args: Up to two optional args that are echoed to stdout

#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running success_sample_args.sh"


# collect args
arg1=$1
arg2=$2

# output args and arg count to stdout
```

```
info "number of args: $#"
info "arg1 ${arg1}"
info "arg2 ${arg2}"

# exit with 0 to indicate success
info "exit 0"
exit 0
```

スナップショット前 / スナップショット後のフックの例

次の例は、 Snapshot 前フックと Snapshot 後フックの両方に同じスクリプトを使用する方法を示しています。

```
#!/bin/sh

# success_sample_pre_post.sh
#
# A simple success hook script example with an arg for testing purposes
# to demonstrate how the same script can be used for both a prehook and
posthook
#
# args: [pre|post]

# unique error codes for every error case
ebase=100
eusage=$((ebase+1))
ebadstage=$((ebase+2))
epre=$((ebase+3))
epost=$((ebase+4))


#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
```

```
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# Would run prehook steps here
#
prehook() {
    info "Running noop prehook"
    return 0
}

#
# Would run posthook steps here
#
posthook() {
    info "Running noop posthook"
    return 0
}


#
# main
#

# check arg
stage=$1
if [ -z "${stage}" ]; then
    echo "Usage: $0 <pre|post>"
    exit ${eusage}
fi

if [ "${stage}" != "pre" ] && [ "${stage}" != "post" ]; then
    echo "Invalid arg: ${stage}"
    exit ${ebadstage}
fi
```

```
# log something to stdout
info "running success_sample_pre_post.sh"

if [ "${stage}" = "pre" ]; then
    prehook
    rc=$?
    if [ ${rc} -ne 0 ]; then
        error "Error during prehook"
    fi
fi

if [ "${stage}" = "post" ]; then
    posthook
    rc=$?
    if [ ${rc} -ne 0 ]; then
        error "Error during posthook"
    fi
fi

exit ${rc}
```

失敗の例

次の例は、フックで障害を処理する方法を示しています。

```
#!/bin/sh

# failure_sample_arg_exit_code.sh
#
# A simple failure hook script for testing purposes.
#
# args: [the exit code to return]
#



#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}



#
```

```
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running failure_sample_arg_exit_code.sh"

argexitcode=$1

# log to stderr
error "script failed, returning exit code ${argexitcode}"

# exit with specified exit code
exit ${argexitcode}
```

詳細なエラーの例

次の例では ' フックの失敗をより詳細なロギングで処理する方法を示します

```
#!/bin/sh

# failure_sample_verbose.sh
#
# A simple failure hook script with args for testing purposes.
#
# args: [The number of lines to output to stdout]
```

```
#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running failure_sample_verbose.sh"


# output arg value to stdout
linecount=$1
info "line count ${linecount}"

# write out a line to stdout based on line count arg
i=1
while [ "$i" -le ${linecount} ]; do
    info "This is line ${i} from failure_sample_verbose.sh"
    i=$(( i + 1 ))
```

```
done

error "exiting with error code 8"
exit 8
```

終了コード例を使用した失敗

次の例は、終了コードを使用したフックの失敗を示しています。

```
#!/bin/sh

# failure_sample_arg_exit_code.sh
#
# A simple failure hook script for testing purposes.
#
# args: [the exit code to return]
#


#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}
```

```
#
# main
#

# log something to stdout
info "running failure_sample_arg_exit_code.sh"

argexitcode=$1

# log to stderr
error "script failed, returning exit code ${argexitcode}"

# exit with specified exit code
exit ${argexitcode}
```

失敗後の成功の例

次の例では、最初の実行時にフックが失敗していますが、 2 回目の実行後に成功しています。

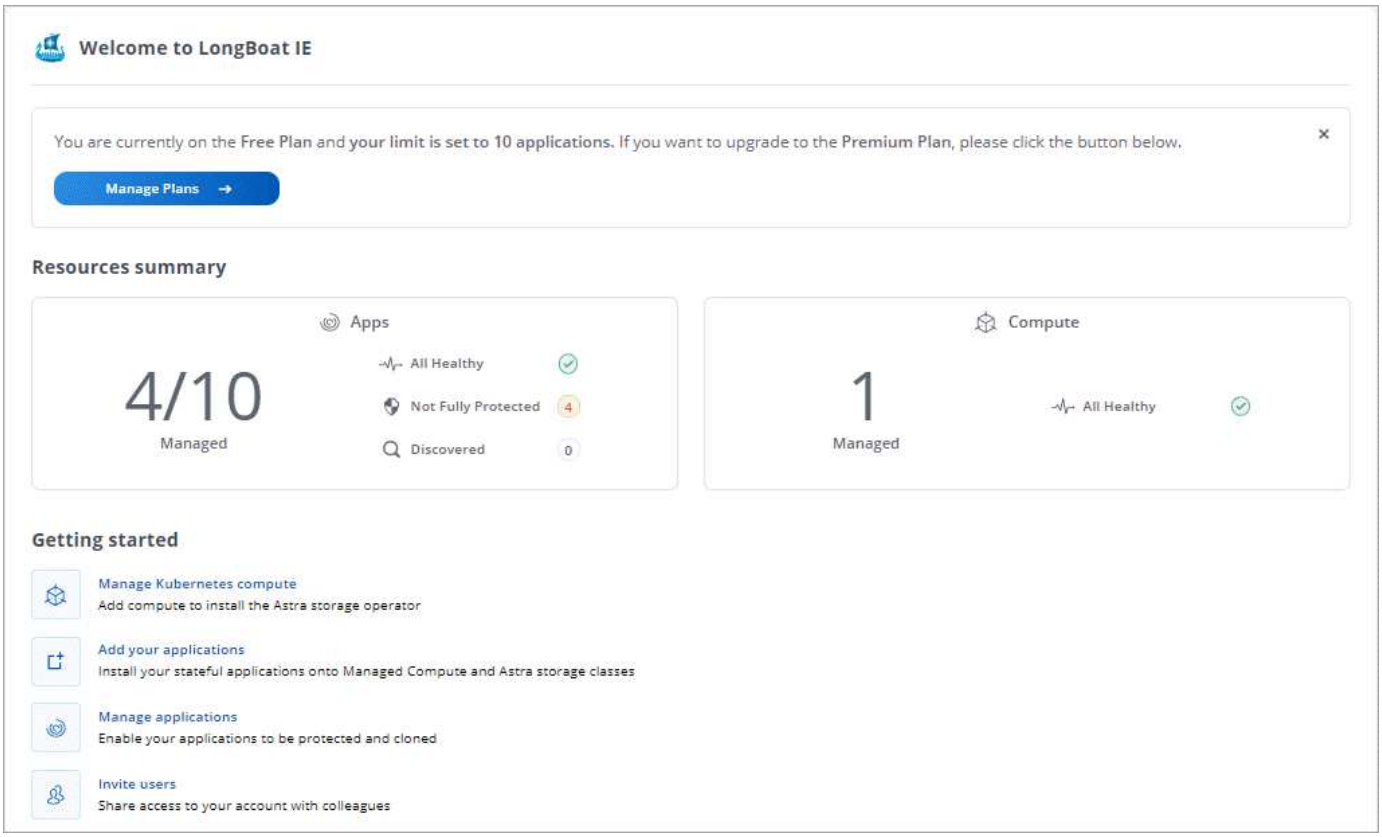
```
#!/bin/sh

# failure_then_success_sample.sh
#
# A hook script that fails on initial run but succeeds on second run for
testing purposes.
#
# Helpful for testing retry logic for post hooks.
#
# args: None
#

#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
```

```
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}



#
# main
#

# log something to stdout
info "running failure_success sample.sh"


if [ -e /tmp/hook-test.junk ] ; then
    info "File does exist.  Removing /tmp/hook-test.junk"
    rm /tmp/hook-test.junk
    info "Second run so returning exit code 0"
    exit 0
else
    info "File does not exist.  Creating /tmp/hook-test.junk"
    echo "test" > /tmp/hook-test.junk
    error "Failed first run, returning exit code 5"
    exit 5
fi
```

アプリとコンピューティングの健全性を表示します

アプリケーションとクラスタの健常性の概要を表示します

ダッシュボード * をクリックすると、アプリ、クラスター、ヘルスの概要が表示されます。

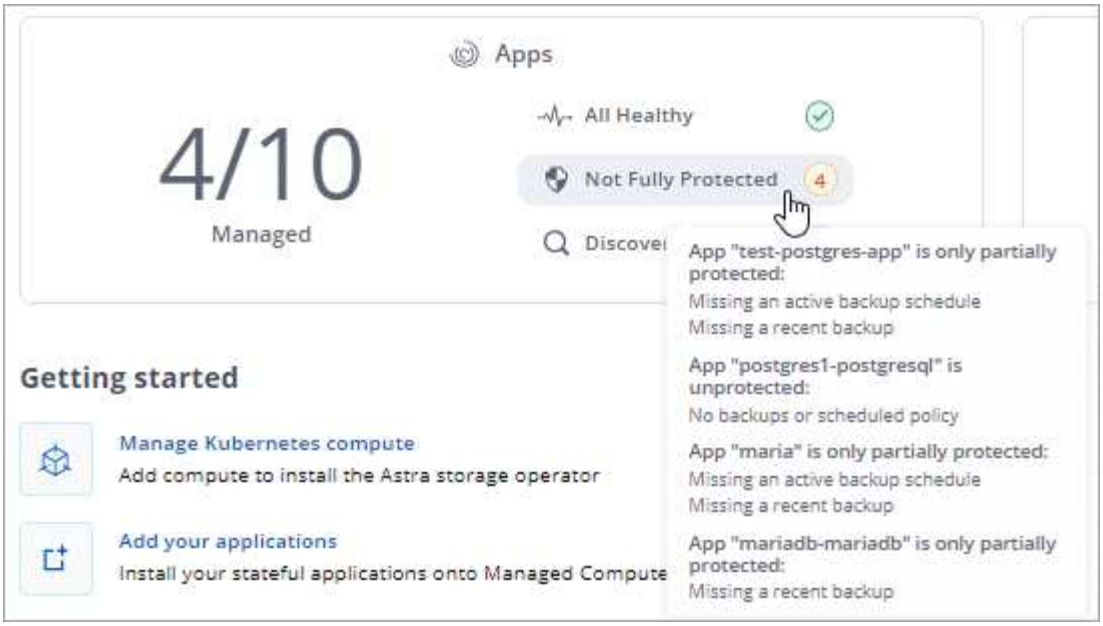


[アプリケーション（ Apps ）] タイルでは、次の項目を識別できます。

- 現在管理しているアプリケーションの数。
- それらの管理アプリが正常であるかどうか。
- アプリケーションが完全に保護されているかどうか（最新のバックアップがある場合は保護されます）。
- 検出されたものの、まだ管理されていないアプリケーションの数。

  アプリケーションが検出された後で管理または無視するため、この数はゼロになるのが理想的です。さらに、ダッシュボードで検出されたアプリケーションの数を監視して、開発者がクラスタに新しいアプリケーションを追加するタイミングを特定します。

これらは数字やステータスだけではなく、それぞれのステータスからドリルダウンすることができます。たとえば、アプリが完全に保護されていない場合は、アイコンの上にカーソルを置くと、完全に保護されていないアプリを特定できます。その理由が含まれます。



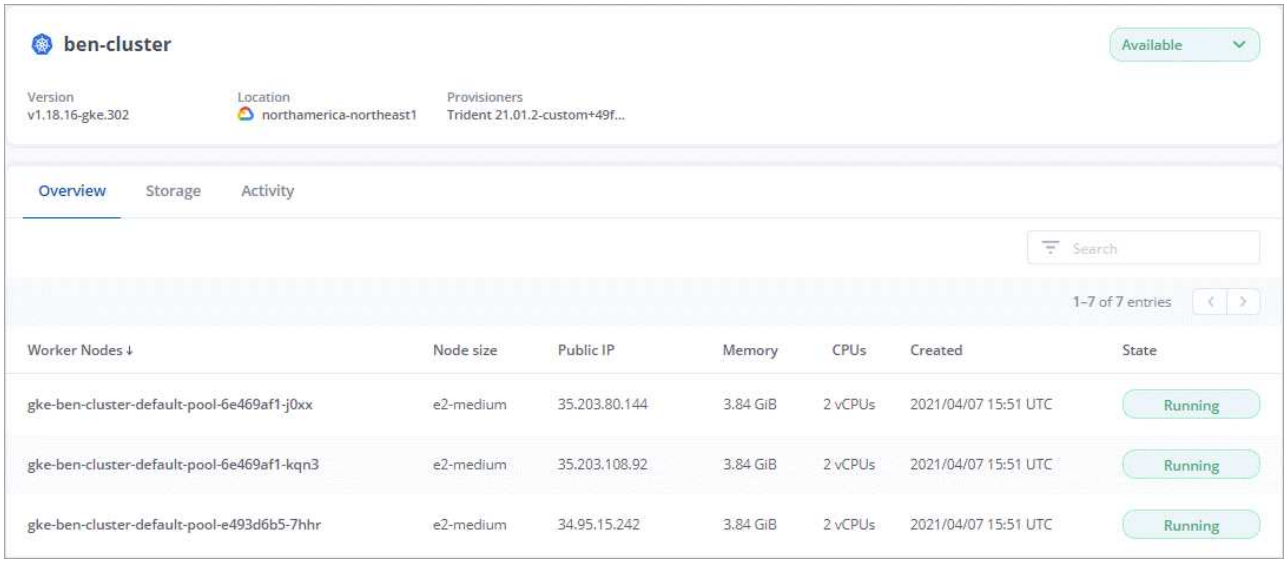
クラスタタイルには、クラスタの健常性に関する同様の詳細が表示され、アプリケーションと同様に詳細を確認できます。

クラスタの健常性と詳細を表示します

Kubernetes クラスタを Astra Control に追加すると、クラスタの場所、ワーカーノード、永続ボリューム、ストレージクラスなどの詳細を表示できます。

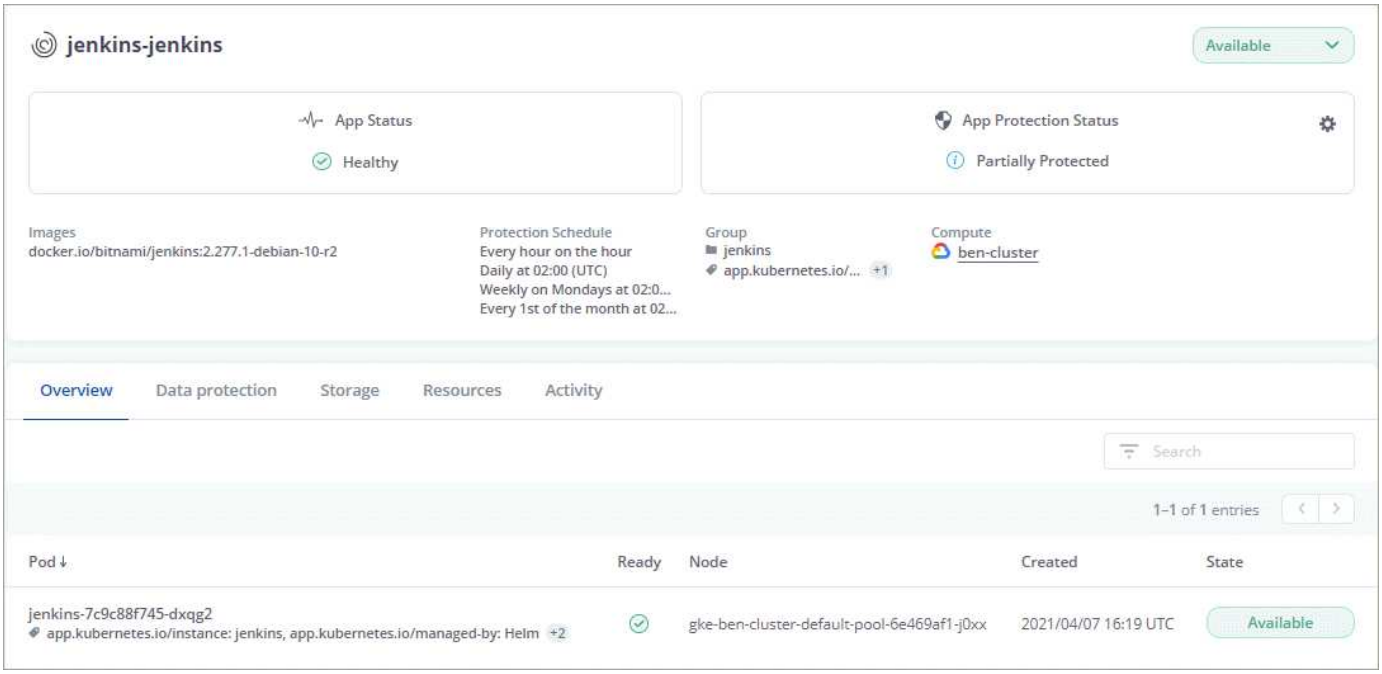
手順

1. 「 * クラスタ * 」を選択します。
2. クラスタ名を選択します。
3. [Overview （概要）] タブと [Storage （ストレージ）] タブで情報を確認し、必要な情報を見つけます。
    - * 概要 * ：状態を含むワーカーノードの詳細。
    - * Storage * ：ストレージクラスと状態を含む、クラスタに関連付けられた永続的ボリューム。
    - * アクティビティ * ：クラスタに関連する Astra のアクティビティ。



アプリの状態と詳細を表示します

アプリケーションの管理を開始すると、アプリケーションのステータス（正常かどうか）、保護ステータス（障害発生時に完全に保護されているかどうか）、ポッド、永続的ストレージなどを識別できる詳細が Astra Control に表示されます。



手順

1. 「 * アプリケーション」を選択し、アプリケーションの名前を選択します。
2. お探しの情報を検索するには、 [検索する] を選択します。

アプリステータス

Kubernetes でのアプリケーションの状態を反映するステータスを提供します。たとえば、ポッドと永続ボリュームはオンラインか？アプリケーションが正常な状態でない場合は、 Kubernetes のログでクラスタの問題を調べてトラブルシューティングする必要があります。Astra Control では、壊れたアプ

リケーションを修正するための情報は提供されません。

アプリ保護ステータス

- * 完全に保護されている * ：アプリにはアクティブなバックアップスケジュールがあり、 1 週間も経過していない正常なバックアップがあります
- * 部分的に保護 * ：アプリケーションには、アクティブなバックアップスケジュール、アクティブなスナップショットスケジュール、または正常なバックアップまたはスナップショットがあります
- * 保護されていない * ：完全に保護されていない、または部分的に保護されていないアプリ

  _ 最新のバックアップがあるまで、完全に保護することはできません _ 。これは、永続ボリュームから離れたオブジェクトストアにバックアップが格納されるために重要です。障害や事故によってクラスタと永続的ストレージが消去された場合は、バックアップをリカバリする必要があります。スナップショットを使用してリカバリすることはできません。

概要

アプリケーションに関連付けられているポッドの状態に関する情報。

データ保護

データ保護ポリシーを設定し、既存の Snapshot とバックアップを表示できます。

ストレージ

アプリケーションレベルの永続ボリュームが表示されます。永続ボリュームの状態は、 Kubernetes クラスタから見たものです。

リソース

バックアップおよび管理対象のリソースを確認できます。

アクティビティ

アプリケーションに関連する Astra Control アクティビティ。

バケットを管理する

Astra がバックアップとクローンに使用するバケットを管理するには、バケットを追加し、クラウドプロバイダ内の Kubernetes クラスタのデフォルトバケットを変更します。

バケットを追加および変更できるのは Admin のみです。

Astra Control でのバケットの使用方法

最初の Kubernetes クラスタの管理を開始すると、 Astra Control Service が、管理対象クラスタと同じ場所にクラウドプロバイダのデフォルトバケットを作成します。

Astra Control Service は、作成したバックアップとクローンにこのデフォルトバケットを使用します。その後、バックアップを使用して、クラスタ間でアプリケーションのリストアやクローニングを行うことができます。

Astra Control Service にバケットを追加する場合は、保護ポリシーの作成時にこれらのバケットから選択できます。また、 Astra Control Service がアドホックバックアップやクローンに使用するデフォルトバケットを変

更することもできます。クラウドプロバイダ間でバケットをクローニングすることはできません。クラウドプロバイダ間でデータを移行するには、関連するアプリケーションのクローンを作成します。

> Astra Control Service は、バックアップまたはクローンを開始する前に、デスティネーションバケットがアクセス可能かどうかをチェックします。

既存のバケットを表示します

Astra Control Service で使用可能なバケットのリストを表示して、クラウドプロバイダのステータスを確認したり、デフォルトのバケットを特定したりできます。

バケットの状態は次のいずれかになります。

保留中です
: バケットを追加すると、最初に Pending 状態から Astra Control が初めてバケットを参照します。

利用可能
: バケットは Astra Control で使用できます。

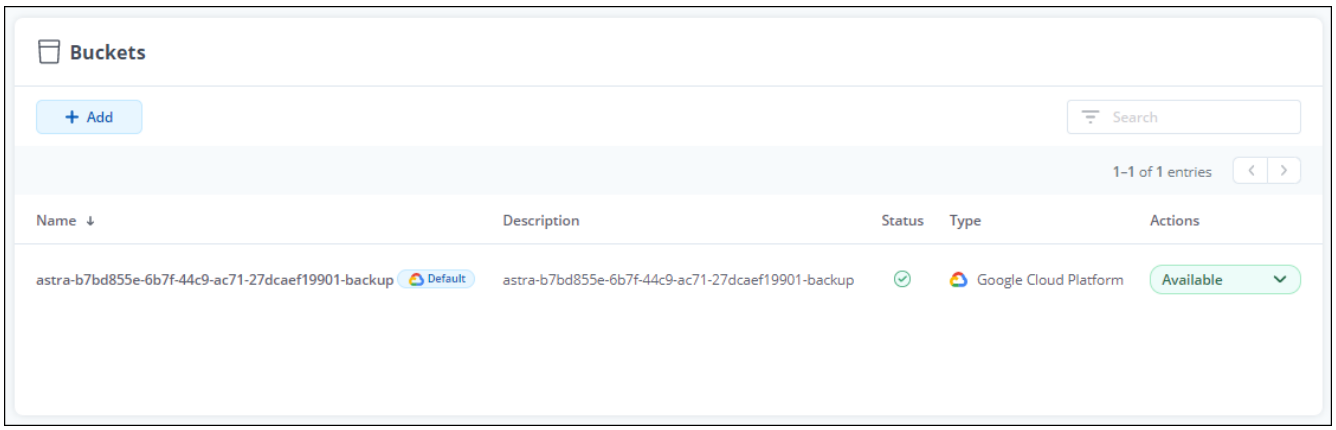
削除されました
: バケットが Removed 状態の場合でも、バケットをデフォルトのバケットとして設定し、保護スケジュールに割り当てることができます。ただし、データ保護処理の開始時点でバケットが「 available 」状態でない場合は、処理が失敗します。

ステップ

1. [* ストレージの管理 *] で、 [* バケット *] を選択します。

   Astra Control Service で使用できるバケットのリストが表示されます。

   次の例からわかるように、使用できるバケットは 1 つだけです。つまり、ネットアップが作成したデフォルトのバケットです。



追加のバケットを追加します

クラウドプロバイダでクラスタの管理を開始したら、いつでもバケットを追加できます。これにより、保護ポリシーの作成時にバケットを選択したり、アドホックバックアップやクローンのデフォルトバケットを変更したりできます。

Astra Control Service では、バケットを追加したあとに削除できないことに注意してください。

必要なもの

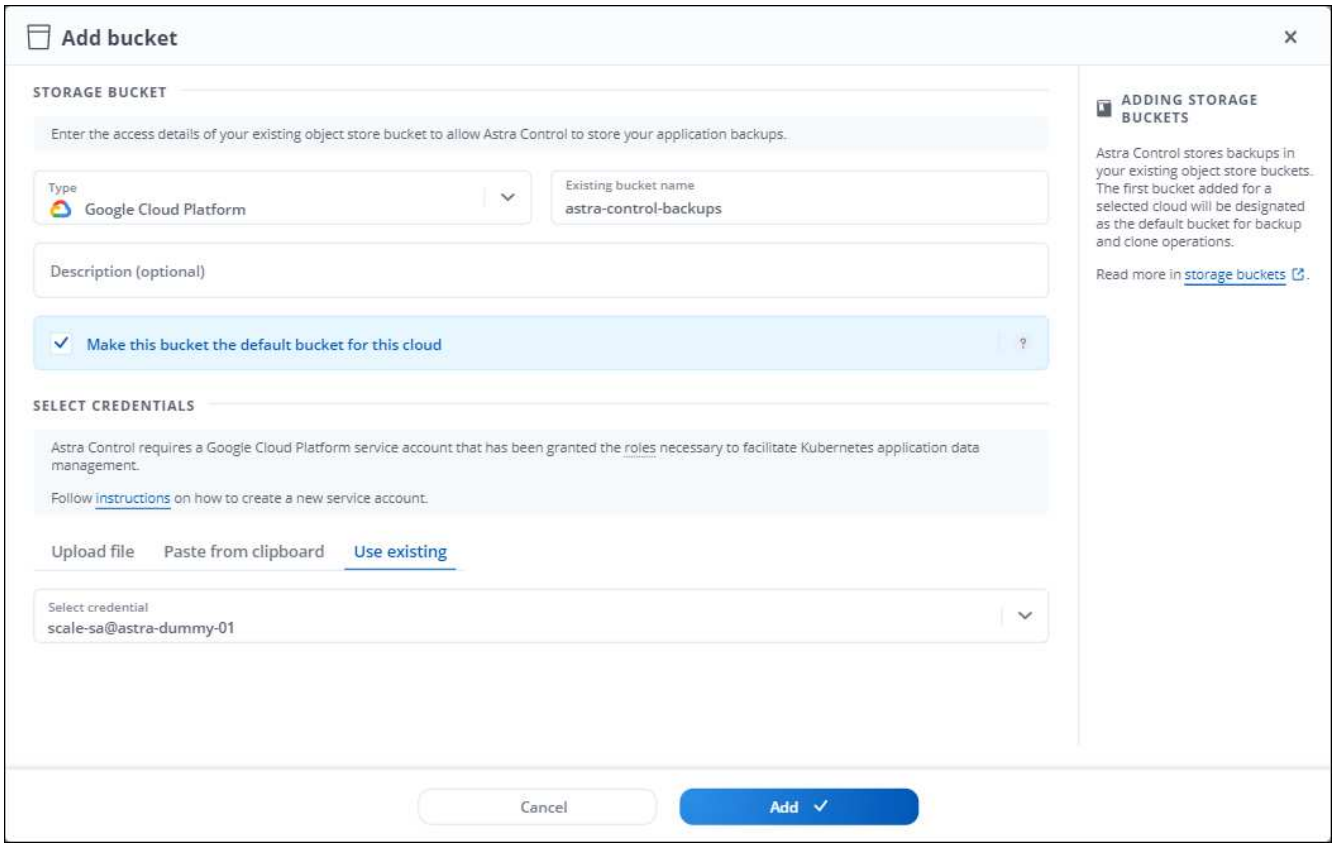
- クラウドプロバイダ内の既存のバケットの名前。
- バケットが Azure 内にある場合は、 *Astra -backup -rg* という名前のリソースグループに属している必要があります。

手順

1. [* ストレージの管理 *] で、 [* バケット *] を選択します。
2. 「 * Add * 」を選択し、プロンプトに従ってバケットを追加します。
   - * タイプ * ：クラウドプロバイダを選択します。

     クラウドプロバイダを利用できるのは、 Astra Control Service がクラウドプロバイダで実行されているクラスタの管理を開始したあとのみです。
   - * 既存のバケット名 * ：バケットの名前を入力します。
   - * 概要 * ：必要に応じて、バケットの概要を入力します。
   - * このバケットをこのクラウドのデフォルトバケットにする * ：このバケットをアドホックバックアップやクローンのデフォルトバケットとして使用するかどうかを選択します。
   - * クレデンシャルを選択 * ：バケットを管理するために必要な権限を Astra Control Service に付与するクレデンシャルを選択します。

     Google Cloud Platform に新しいバケットを追加する例を次に示します。



3. 「 * Add * 」を選択してバケットを追加します。

Astra Control Service は、追加のバケットを追加します。保護ポリシーの作成時にバケットを選択できるようになりました。

デフォルトバケットを変更する

バックアップとクローンに Astra Control Service で使用するデフォルトバケットを変更する。各クラウドプロバイダには、独自のデフォルトバケットがあります。

Astra Control Service では、クラウドプロバイダ用のデフォルトバケットを使用して、既存のバックアップからクローニングすることを選択しなかった場合に、アドホックバックアップやアドホッククローンを作成します。

手順

1. [* ストレージの管理 *] で、 [* バケット *] を選択します。
2. 編集するバケットの「 * Actions * 」列でドロップダウンリストを選択します。
3. このバケットをこのクラウドのデフォルトバケットにする * を選択します。
4. 「 * Update * 」を選択します。

アカウントを管理します

請求を設定します

Astra Control の無料プランを使用すると、最大 10 個のアプリをアカウントで管理できます。10 以上のアプリを管理する場合は、無料プランからプレミアムプランにアップグレードして請求を設定する必要があります。

課金の概要

Astra Control Service の使用には、ネットアップの Astra Control Service の料金と、永続的ボリュームとオブジェクトストレージをクラウドプロバイダから課金する、 2 種類のコストが関連します。

Astra Control サービスの請求

Astra Control Service には、次の 3 つのプランがあります。

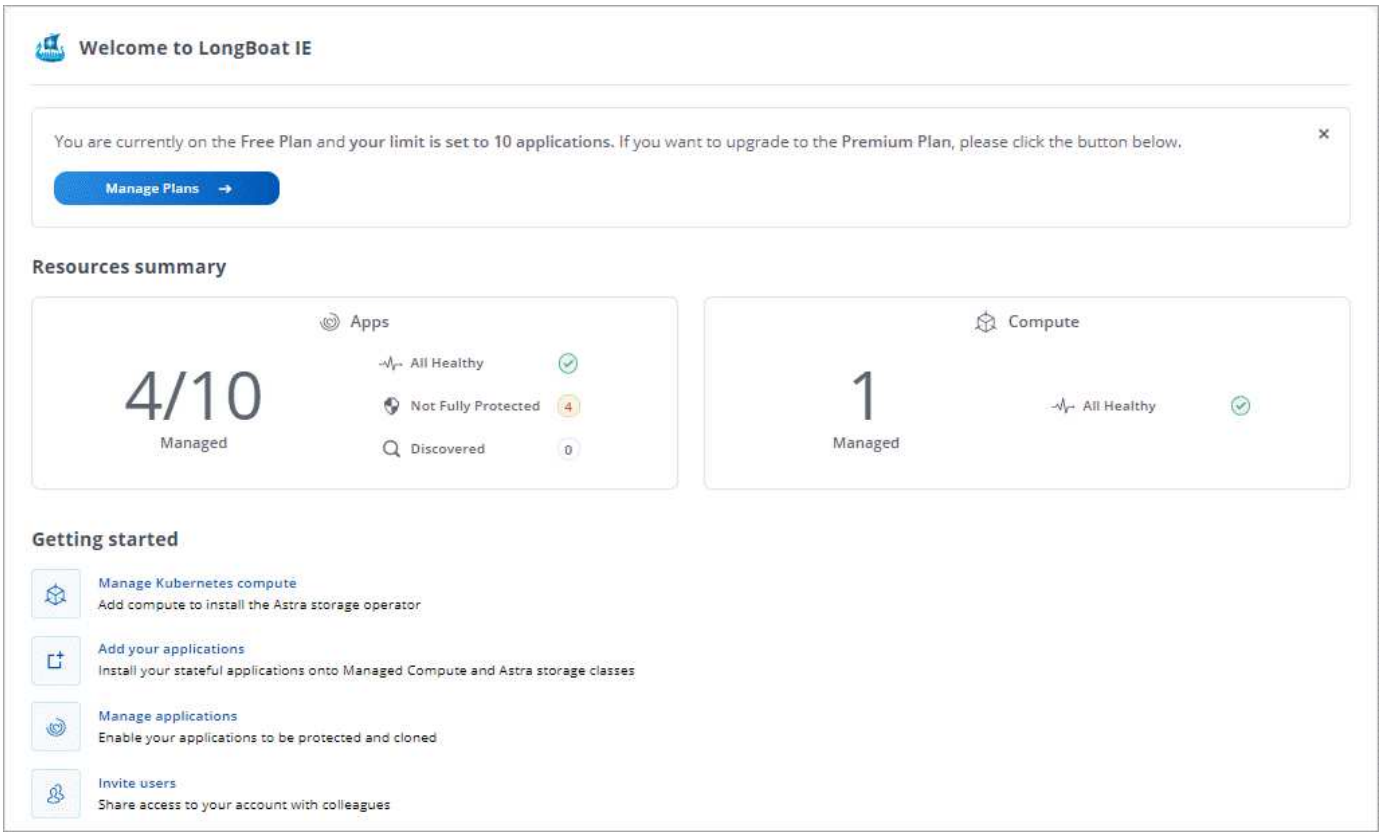
自由な計画
: 最大 10 個のアプリを無料で管理できます。

Premium PayGo の場合
: アプリごとに 1 分あたり $0.005 の割合で、無制限のアプリを管理します。

プレミアムサブスクリプション
: まだ Astra Control アカウントをお持ちでない場合は、 Premium サブスクリプションを購入すると Astra Control アカウントが自動的に作成されます。既存の無料プランがある場合は、自動的に Premium サブスクリプションに変換されます。

Astra Control アカウントを作成すると、無料プランに自動的にサインアップされます。Astra Control のダッ

シュボードには、現在管理している 10 個の無料アプリのうち、許可されているものをいくつ管理しているかが表示されます。



11 番目のアプリを管理しようとすると、 Astra Control から無料プランの制限に達したことが通知されます。その後、フリープランからプレミアムプランへのアップグレードを求めるプロンプトが表示されます。

Premium プランにはいつでもアップグレードできます。アップグレード後、アカウントの _all_managed アプリの料金が Astra Control によって請求されます。最初の 10 個のアプリは、無料プランには含まれていません。

Google Cloud の請求

Astra Control Service を使用して GKE クラスタを管理する場合、永続ボリュームは NetApp Cloud Volumes Service によってバックアップされ、アプリのバックアップは Google Cloud Storage バケットに保存されます。

- "Cloud Volumes Service の価格設定の詳細を表示"。

  Astra Control Service は、すべてのサービスタイプとサービスレベルをサポートします。使用するサービスタイプは、によって異なります "Google Cloud リージョン"。

- "Google Cloud ストレージバケットの価格設定の詳細を表示します"。

Microsoft Azure 請求

AKS クラスタを Astra Control Service で管理する場合、永続ボリュームは Azure NetApp Files によってバックアップされ、アプリケーションのバックアップは Azure BLOB コンテナに格納されます。

- "Azure NetApp Files の価格設定の詳細を表示"。

- "Microsoft Azure BLOB ストレージの価格設定の詳細を表示する"。

重要事項

- 請求計画は Astra Control アカウントごとに作成します。

  アカウントが複数ある場合は、それぞれに独自の課金プランがあります。

- Astra Control の料金には、 Kubernetes アプリケーションの管理費用が含まれています。永続ボリュームのストレージバックエンド用に、クラウドプロバイダから別途料金が請求されます。

  "Astra Control の価格設定の詳細"。

- 各請求期間は、その月の最終日に終了します。
- プレミアムプランからフリープランにダウングレードすることはできません。

無料プランから **Premium PayGo** プランへのアップグレード

支払いプランをいつでもアップグレードして、必要に応じて料金を支払うことで、 Astra Control から 10 以上のアプリの管理を開始できます。必要なのは有効なクレジットカードだけです。

手順

1. [*Account] を選択し、 [*Billing *] を選択します。
2. [* プラン *] で、 [* Premium PayGo*] に移動し、 [* 今すぐアップグレード *] を選択します。
3. 有効なクレジットカードの支払い情報を入力して、「プレミアムプランへのアップグレード」を選択します。

> クレジットカードの有効期限が近づいている場合は、 Astra Control から電子メールでお知らせします。

10 以上のアプリを管理できるようになりました。Astra Control は、現在管理している _ALL_APPS の課金を開始します。

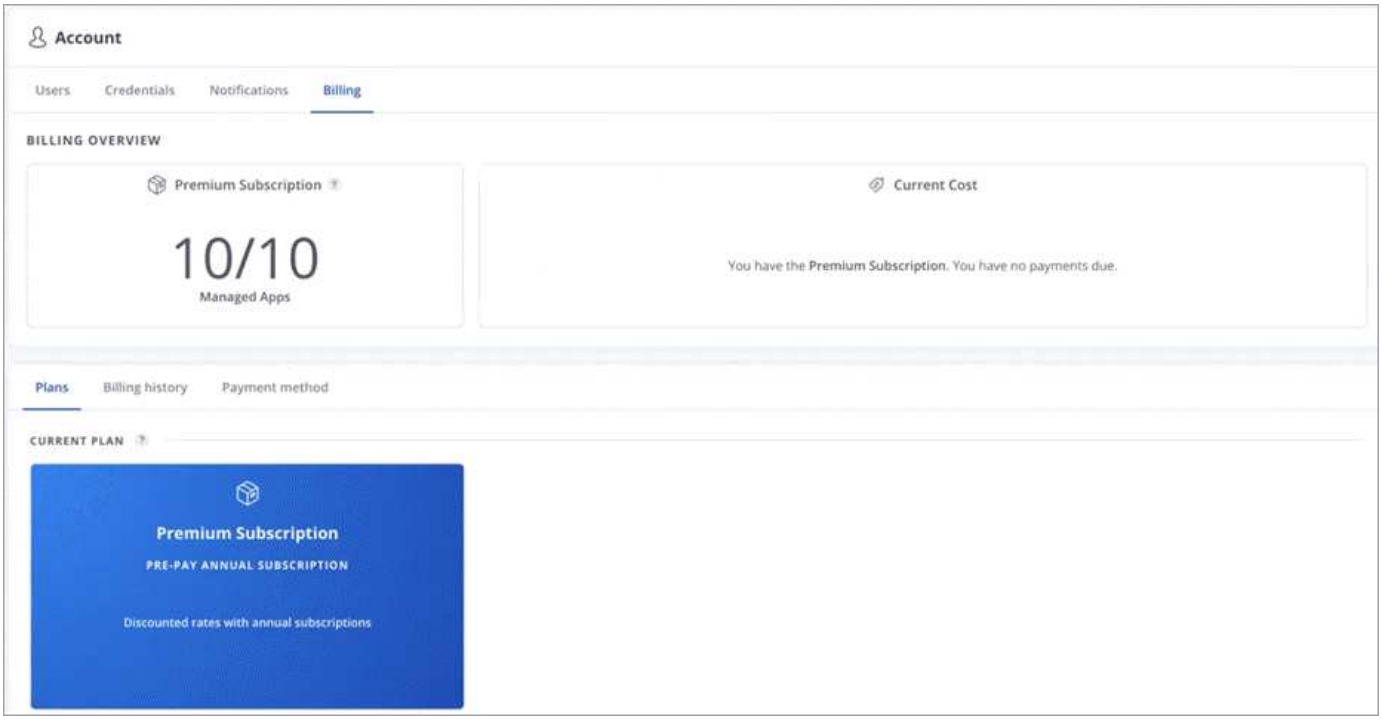
無料プランからプレミアムサブスクリプションへのアップグレード

ネットアップの営業担当に問い合わせて、年単位のサブスクリプション料金が割引価格で前払いされていることを確認してください。

手順

1. [*Account] を選択し、 [*Billing *] を選択します。
2. [* プラン *] で、 [* プレミアムサブスクリプション *] に移動し、 [* セールスへのお問い合わせ] を選択します。
3. プロセスを開始するための詳細を営業チームに入力します。

発注プロセスを進める際には、ネットアップの営業担当者からご連絡いたします。注文が完了すると、 Astra Control は請求タブに現在の計画を反映します。



現在のコストと請求履歴を表示します

Astra Control には、現在の月単位のコストと、アプリケーションごとの詳細な請求履歴が表示されます。

手順

1. [*Account] を選択し、 [*Billing *] を選択します。

   現在のコストは、課金の概要の下に表示されます。

2. アプリで請求履歴を表示するには、「請求履歴」を選択します。

   Astra Control を使用すると、各アプリケーションの使用時間とコストを分単位で表示できます。使用分は、課金期間中に Astra Control がアプリケーションを管理した分数です。

3. ドロップダウンリストを選択して前の月を選択します。

Premium PayGo のクレジットカードを変更します

必要に応じて、支払い用に Astra Control のクレジットカードをファイルに変更できます。

手順

1. ［ * アカウント ］ > ［ 請求 ］ > ［ 支払方法 * ］ の順に選択します。
2. 設定アイコンを選択します。
3. クレジットカードを変更します。

ユーザーを招待および削除します

ユーザーを Astra Control アカウントに招待し、アカウントにアクセスできなくなったユーザーを削除します。

ユーザーを招待します

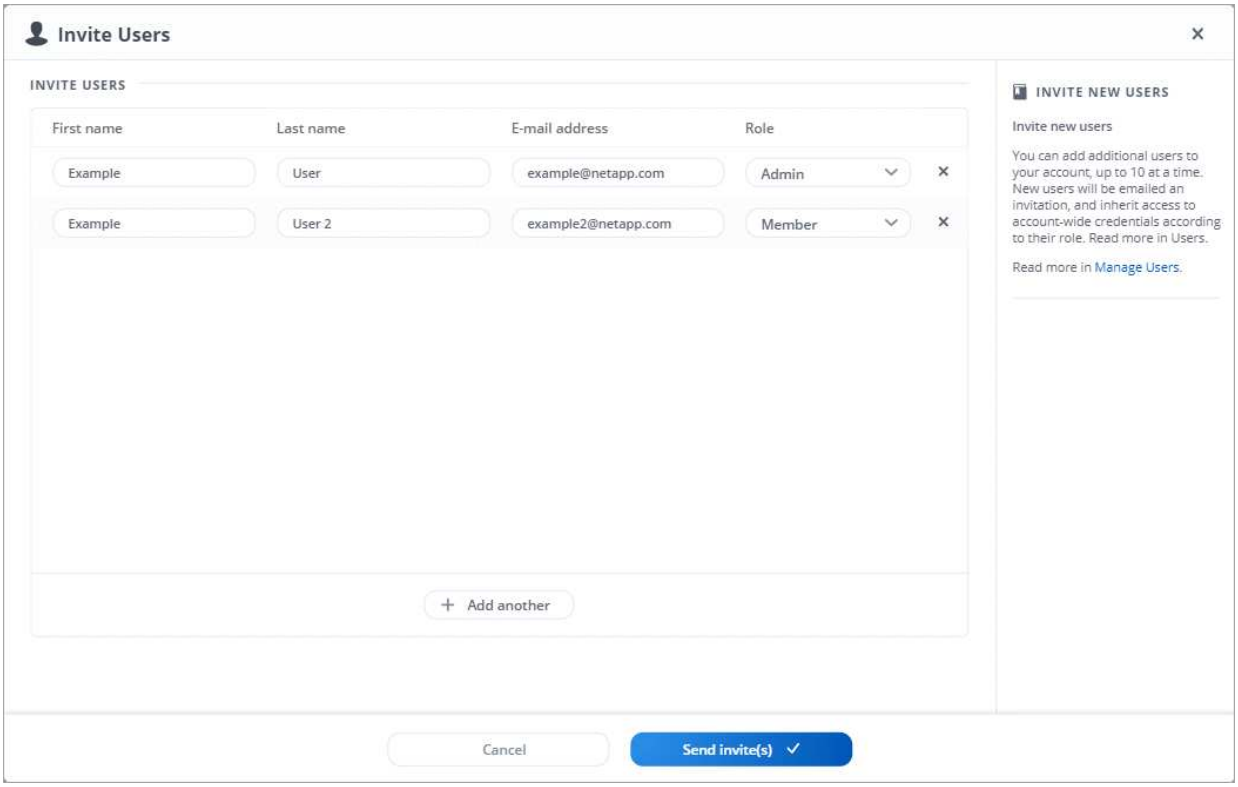
アカウント所有者と管理者は、他のユーザを Astra Control アカウントに招待できます。

手順

1. ユーザにが割り当てられていることを確認します "Cloud Central へのログイン"。
2. 「 * アカウント * 」を選択します。
3. [* ユーザー *] タブで、 [* + ユーザーの招待 *] を選択します。
4. ユーザの名前、 E メールアドレス、およびロールを入力します。

   次の点に注意してください。

   - E メールアドレスが、 Cloud Central へのサインアップに使用した E メールアドレスと一致している必要があります。
   - 各ロールには次の権限があります。
     - * 所有者 * には管理者権限があり、アカウントを削除できます。
     - * Admin * にはメンバー権限があり、他のユーザーを招待できます。
     - メンバー * は、アプリとクラスターを完全に管理できます。
     - * Viewer * はリソースを表示できます。



5. 「 * 招待状を送信する * 」を選択します。

ユーザーには、アカウントへの参加を招待する電子メールが送信されます。

ユーザーの役割を変更します

Account Owner はすべてのユーザのロールを変更できますが、 Account Admin は、 Admin 、 Member 、 Viewer のいずれかのロールを持つユーザのロールを変更できます。

手順

1. 「 * アカウント * 」を選択します。
2. [**Users**] タブで、ユーザーの [*Role] 列のドロップダウンリストを選択します。
3. 新しいロールを選択し、プロンプトが表示されたら ［ * 役割の変更 * ］ を選択します。

Astra Control は、選択した新しいロールに基づいてユーザーの権限を更新します。

ユーザを削除します

アカウント所有者は、アカウントから他のユーザーをいつでも削除できます。

手順

1. 「 * アカウント * 」を選択します。
2. [* ユーザー *] タブで、削除するユーザーを選択します。
3. アクション * を選択し、 * ユーザー / 秒を削除 * を選択します。
4. プロンプトが表示されたら、ユーザー名を入力して削除を確認し、「 * はい、ユーザーを削除 * 」を選択します。

Astra Control を使用すると、アカウントからユーザが削除されます。

クレデンシャルを追加および削除します

クラウドプロバイダのクレデンシャルをアカウントにいつでも追加、削除できます。Astra Control は、Kubernetes クラスタとクラスタ上のアプリケーションを検出し、ユーザに代わってリソースをプロビジョニングするために、このクレデンシャルを使用します。

Astra Control のすべてのユーザが同じクレデンシャルセットを共有していることに注意してください。

クレデンシャルを追加する

Astra Control にクレデンシャルを追加する最も一般的な方法は、クラスタを管理する場合ですが、アカウントページからクレデンシャルを追加することもできます。その後、 Kubernetes クラスタを追加で管理する際にクレデンシャルを選択できるようになります。

必要なもの

- GKE の場合は、必要な権限を持つサービスアカウントのサービスアカウントキーファイルが必要です。 "サービスアカウントの設定方法について説明します"。
- AKS の場合、サービスプリンシパルの作成時に Azure CLI からの出力を含む JSON ファイルが必要です。 "サービスプリンシパルの設定方法について説明します"。

  JSON ファイルに追加していない場合は、 Azure サブスクリプション ID も必要になります。

手順

1. [*Account] > [Credentials] を選択します。
2. [Add Credentials] を選択します。
3. Microsoft Azure * または * Google Cloud Platform * のいずれかを選択します。
4. Astra Control の他のクレデンシャルと区別するクレデンシャルの名前を入力します。
5. 必要なクレデンシャルを入力します。
   a. * Microsoft Azure * ： JSON ファイルをアップロードするか、クリップボードから JSON ファイルの内容を貼り付けて、 Azure サービスプリンシパルの詳細を Astra Control に伝えます。

      JSON ファイルに、サービスプリンシパルの作成時に Azure CLI からの出力が含まれている必要があります。サブスクリプション ID を含めることで、 Astra Control に自動的に追加されるようにすることもできます。ID が必要な場合は、 JSON の指定後に手動で ID を入力する必要があります。

   b. * Google Cloud Platform * ：ファイルをアップロードするか、クリップボードからコンテンツを貼り付けて、 Google Cloud サービスアカウントのキーファイルを提供します。
6. [Add Credentials] を選択します。

これで、 Astra Control にクラスタを追加するときに、クレデンシャルを選択できるようになりました。

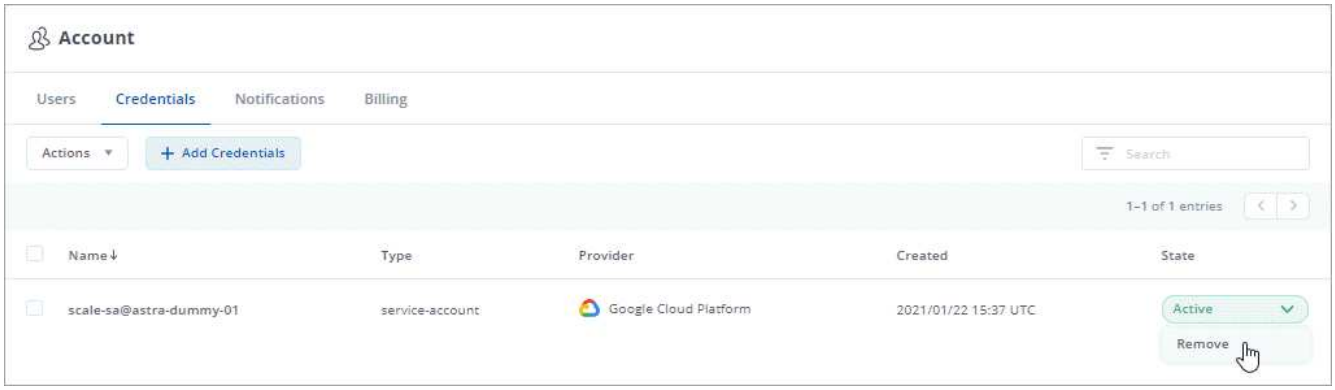
クレデンシャルを削除する

アカウントからのクレデンシャルの削除はいつでも実行できます。クレデンシャルは、のあとに削除してください "すべてのクラスタの管理を解除します"。

> Astra Control に追加する最初のクレデンシャルセットは常に使用されています。 Astra Control は、そのクレデンシャルを使用してバックアップバケットへの認証を行うためです。これらのクレデンシャルは削除しないことを推奨します。

手順

1. [*Account] > [Credentials] を選択します。
2. 削除するクレデンシャルの「 * State * 」列でドロップダウンリストを選択します。
3. 「 * 削除」を選択します。



ページの [資格情報] タブのスクリーンショットで、状態列を選択して [削除] アクションを選択できます。"]

4. 削除を確認するために資格情報の名前を入力し、 * はい、資格情報を削除 * を選択します。

Astra Control は、アカウントからクレデンシャルを削除します。

アカウントアクティビティを表示します

Astra Control アカウントのアクティビティの詳細を表示できます。たとえば、新しいユーザを招待したとき、クラスタが追加されたとき、 Snapshot が作成されたときなどです。アカウントアクティビティを CSV ファイルにエクスポートすることもできます。

Astra Control のすべてのアカウントアクティビティを表示する手順

1. 「 * Activity * 」を選択します。
2. フィルタを使用してアクティビティのリストを絞り込むか、検索ボックスを使用して探しているものを正確に検索します。
3. アカウントアクティビティを CSV ファイルにダウンロードするには、「 * CSV にエクスポート」を選択します。

特定のアプリケーションのアカウントアクティビティを表示する手順

1. 「 * アプリケーション」を選択し、アプリケーションの名前を選択します。
2. 「 * Activity * 」を選択します。

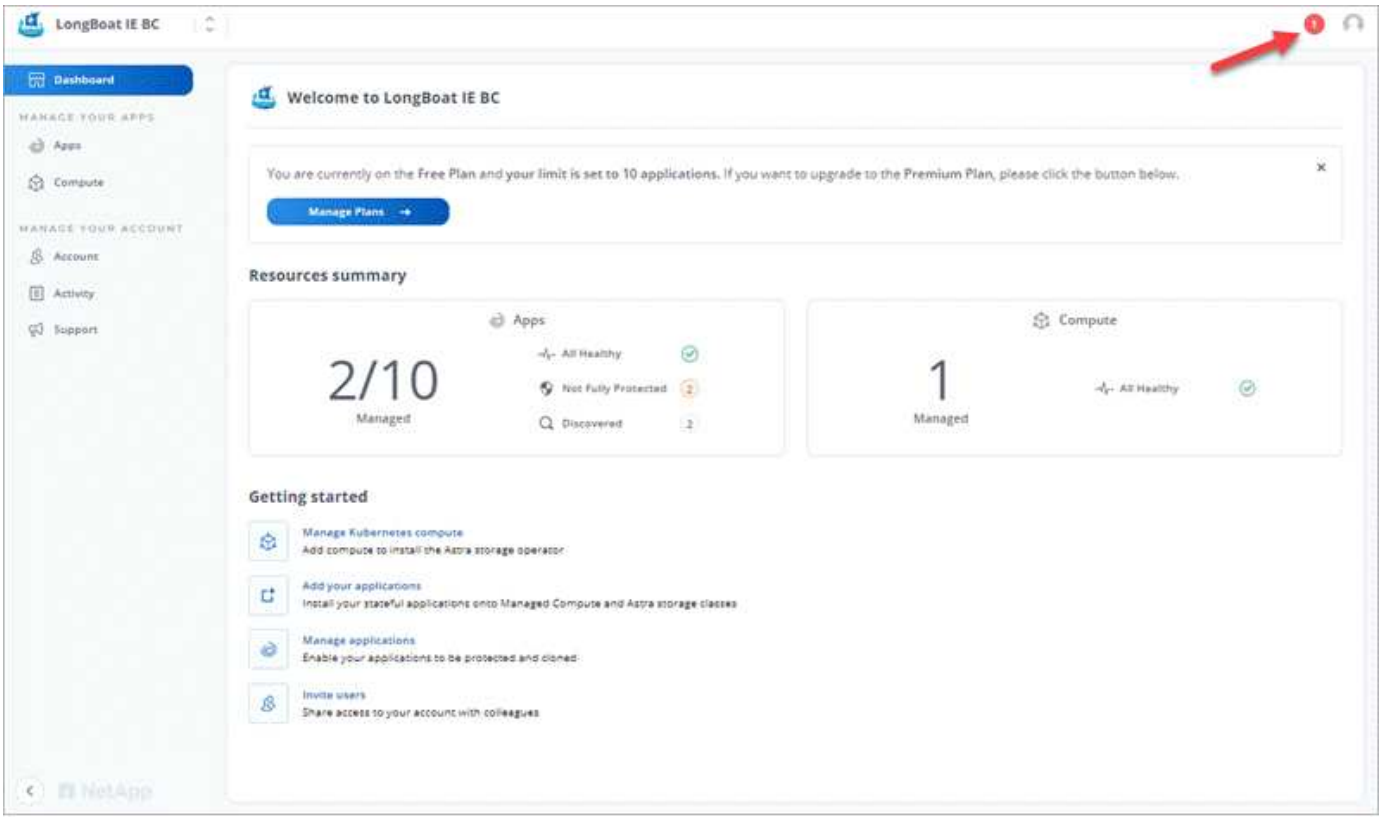
クラスタのアカウントアクティビティを表示する手順

1. 「 * クラスタ」を選択し、クラスタの名前を選択します。
2. 「 * Activity * 」を選択します。

通知を表示および管理します

アクションが完了または失敗すると、 Astra Control から通知が表示されます。たとえば、アプリケーションのバックアップが正常に完了した場合に通知が表示されます。

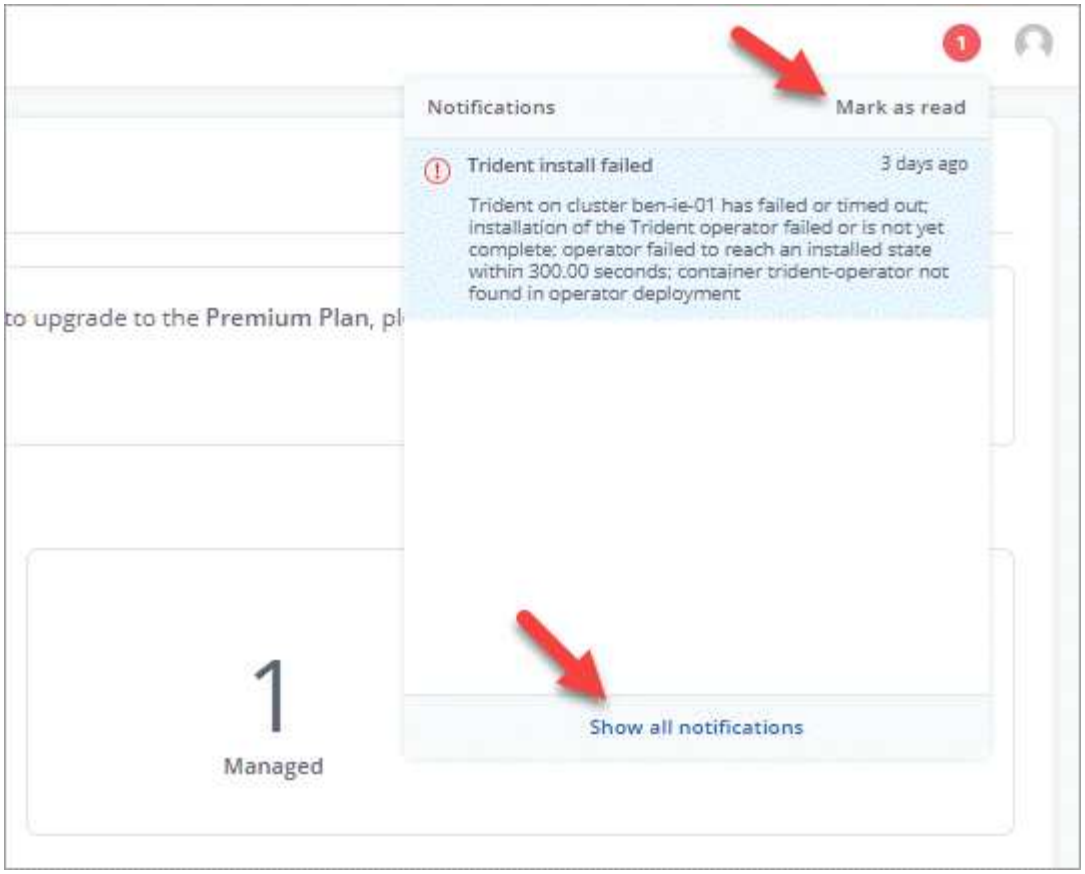
未読通知の数は、インターフェイスの右上に表示されます。



これらの通知を表示して、既読としてマークすることができます（未読の通知を消去する場合などに便利です）。

手順

1. 右上の未読通知の数を選択します。



2. 通知を確認し、 [* 既読としてマークする *] または [すべての通知を表示する *] を選択します。

   [すべての通知を表示する *] を選択した場合は、 [通知] ページがロードされます。

3. [* 通知 *] ページで、通知を表示し、既読としてマークする通知を選択し、 [* アクション *] を選択して、 [* 既読としてマークする *] を選択します。

アカウントを閉じます

Astra Control アカウントが不要になった場合は、いつでもアカウントを閉じることができます。

> Astra Control が自動的に作成したバケットは、アカウントを閉じると自動的に削除されます。

手順

1. "すべてのアプリケーションとクラスタの管理を解除します"。
2. "Astra Control からクレデンシャルを削除"。
3. ［ * アカウント ］ > ［ 請求 ］ > ［ 支払方法 * ］ の順に選択します。
4. 「アカウントを閉じる」を選択します。
5. アカウント名を入力して確定し、アカウントを閉じます。

アプリケーションとクラスタの管理を解除します

管理する必要がなくなったアプリケーションやクラスタを Astra Control から削除

アプリの管理を停止します

バックアップ、スナップショット、またはクローンを作成する必要がなくなったアプリケーションの管理を Astra Control から停止します。

- 既存のバックアップと Snapshot がすべて削除されます。
- アプリケーションとデータは引き続き使用できます。

手順

1. 「 * アプリケーション * 」を選択します。
2. 管理する必要がなくなったアプリのチェックボックスをオンにします。
3. [* アクション * （ * Action * ）] ドロップダウンを選択し、 [* アプリケーションの管理解除 * （ Unmanage application/s * ）] を選択します。
4. アプリの管理を解除することを確認し、 [はい] 、 [アプリケーションの管理を解除] を選択します。

Astra Control がアプリケーションの管理を停止。

クラスタの管理を停止します

管理する必要がなくなったクラスタを Astra Control から管理しないようにします。ベストプラクティスとして、 GCP からクラスタを削除する前に、 Astra Control からクラスタを削除することを推奨します。

- この操作を実行すると、 Astra Control でクラスタが管理されなくなります。クラスタの構成は変更されず、クラスタも削除されません。
- Astra Trident はクラスタからアンインストールされません。 "Astra Trident のアンインストール方法をご確認ください"。

手順

1. 「 * クラスタ * 」を選択します。
2. 管理する必要がなくなったクラスタのチェックボックスを選択します。
3. [* アクション * （ Actions * ）] ドロップダウンを選択し、 [* 管理解除 * （ Unmanage * ）] を選択します
4. クラスタの管理を解除することを確認し、「 * Yes 、 Unmanage cluster * 」を選択します。

Astra Control がクラスタの管理を停止。

クラウドプロバイダからクラスタを削除しています

ネットアップストレージクラスに永続ボリューム（ PV ）が存在する Kubernetes クラスタを削除する前に、次のいずれかの方法で永続ボリューム要求（ PVC ）を削除する必要があります。クラスタを削除する前に PVC と PV を削除すると、クラウドプロバイダから予期しない請求書が届かなくなります。

- * 方法 1 * ：アプリケーションワークロードのネームスペースをクラスタから削除します。Trident ネームスペースを削除しないでください。
- * 方法 #2 * ： PVC とポッド、または PVS がマウントされている配置を削除します。

Kubernetes クラスタを Astra Control から管理する場合、そのクラスタ上のアプリケーションは、永続的ボリ

ュームのストレージバックエンドとして Cloud Volumes Service または Azure NetApp Files を使用します。事前に PVS を削除せずにクラウドプロバイダからクラスタを削除した場合、バックエンドボリュームはクラスタと一緒に _not_deleted になります。

上記のいずれかの方法を使用すると、対応する PVS がクラスタから削除されます。クラスタを削除する前に、クラスタのネットアップストレージクラスに PVS が存在していないことを確認してください。

クラスタを削除する前に永続ボリュームを削除していない場合は、 Cloud Volumes Service for Google Cloud または Azure NetApp Files からバックエンドボリュームを手動で削除する必要があります。

Copyright Information

Copyright © 2022 NetApp, Inc. All rights reserved. Printed in the U.S. No part of this document covered by copyright may be reproduced in any form or by any means-graphic, electronic, or mechanical, including photocopying, recording, taping, or storage in an electronic retrieval system-without prior written permission of the copyright owner.

Software derived from copyrighted NetApp material is subject to the following license and disclaimer:

THIS SOFTWARE IS PROVIDED BY NETAPP “AS IS” AND WITHOUT ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE, WHICH ARE HEREBY DISCLAIMED. IN NO EVENT SHALL NETAPP BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NetApp reserves the right to change any products described herein at any time, and without notice. NetApp assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by NetApp. The use or purchase of this product does not convey a license under any patent rights, trademark rights, or any other intellectual property rights of NetApp.

The product described in this manual may be protected by one or more U.S. patents, foreign patents, or pending applications.

RESTRICTED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.277-7103 (October 1988) and FAR 52-227-19 (June 1987).

Trademark Information

NETAPP, the NETAPP logo, and the marks listed at http://www.netapp.com/TM are trademarks of NetApp, Inc. Other company and product names may be trademarks of their respective owners.